



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER SYSTEMS

ÚSTAV POČÍTAČOVÝCH SYSTÉMŮ

**AUTOMATED DESIGN OF CONSTRUCTIVE HEURIS-
TICS FOR THE VEHICLE ROUTING PROBLEM**

AUTOMATIZOVANÝ NÁVRH KONSTRUKTIVNÍCH HEURISTIK PRO PROBLÉM SMĚROVÁNÍ

VOZIDEL

MASTER'S THESIS

DIPLOMOVÁ PRÁCE

AUTHOR

AUTOR PRÁCE

Bc. ROSTISLAV ČERVENKA

SUPERVISOR

VEDOUCÍ PRÁCE

Ing. DAVID SEDLÁK

BRNO 2026

Master's Thesis Assignment



166391

Institut: Department of Computer Systems (DCSY)
Student: **Červenka Rostislav, Bc.**
Programme: Information Technology and Artificial Intelligence
Specialization: Machine Learning
Title: **Automated Design of Constructive Heuristics for the Vehicle Routing Problem**
Category: Artificial Intelligence
Academic year: 2025/26

Assignment:

1. Familiarise yourself with the Vehicle Routing Problem (VRP) and its variants.
2. Study hyper-heuristics and hyper-heuristic approaches for automated design of constructive heuristics for VRP.
3. Select multiple (at least 4) VRP variants (e.g., CVRP, VRPTW, VRPPD, MDVRP)
4. Using the knowledge gained in assignment items 1 and 2, design and implement a system enabling automated design of constructive heuristics for VRP with properties based on any combination of variants selected in assignment item 3.
5. Experiment with the implemented system, compare the achieved result with those available in the literature.
6. Discuss the achieved results and possible further continuation of the work.

Literature:

- Pillay, N. and Qu, R. Hyper-Heuristics: Theory and Applications. 1st ed. 2018.th ed. Netherlands: Springer Nature, 2018. Natural Computing Series. ISBN 9783319965147
- Jakobović, D.; Đurasević, M.; Brkić, K.; Fosin, J.; Carić, T. et al. Evolving Dispatching Rules for Dynamic Vehicle Routing with Genetic Programming. Algorithms. Basel: MDPI AG, 2023, vol. 16, no. 6, p. 285. ISSN 1999 4893
- Rocha Leite, C. F. M. da. Genetic Programming Approaches for Solving Transportation Problems. ProQuest Dissertations & Theses, 2022
- Gil Gala, F. J.; Đurasević, M. and Jakobović, D. Evolving routing policies for electric vehicles by means of genetic programming. Applied intelligence (Dordrecht, Netherlands). New York: Springer US, 2024, vol. 54, no. 23, p. 12391–12419. ISSN 0924-669X
- Konstantakopoulos, G. D.; Gayialis, S. P. and Kechagias, E. P. Vehicle routing problem and related algorithms for logistics distribution: a literature review and classification. Operational Research, Jul 2022, vol. 22, no. 3, p. 2033–2062. ISSN 1866-1505. Available at: <https://doi.org/10.1007/s12351-020-00600-7>

Requirements for the semestral defence:

- Assignment items 1 to 3. Item 4 in progress.

Detailed formal requirements can be found at <https://www.fit.vut.cz/study/theses/>

Supervisor: **Sedlák David, Ing.**
Head of Department: Sekanina Lukáš, prof. Ing., Ph.D.
Beginning of work: 1.11.2025
Submission deadline: 20.5.2026
Approval date: 4.5.2026

Abstract

This thesis focuses on the automated creation of constructive heuristics for the Vehicle Routing Problem and its variants using a Genetic Programming Hyper-Heuristic approach. A system was developed to evolve routing-priority functions for 16 problem variants that incorporate multiple depots, capacity restrictions, time windows, and electric-vehicle range constraints. Evaluation revealed that the evolved heuristics outperformed the Nearest Neighbor heuristic and heuristics generated using a Large Language Model across all variants, with the most significant performance gains observed in variants that included time windows and range constraints. The generated heuristics also surpassed the Clarke-Wright Savings algorithm in three variants, demonstrating that this approach is an effective alternative to classical construction methods, though with room for further improvement, as shown by comparisons against known optimal solutions.

Abstrakt

Tato práce se zabývá automatickou tvorbou konstruktivních heuristik pro problém směrování vozidel a pro jeho varianty za pomoci hyperheuristického přístupu založeného na genetickém programování. Byl vyvinut systém, který vyvíjí funkce zaměřené na prioritu směrování pro 16 variant tohoto problému. Tyto varianty zahrnují více dep, omezení kapacity, časová okna a omezení dojezdu u elektromobilů. Ohodnocení výsledků ukázalo, že vyvinuté heuristiky dosáhly lepších výsledků než heuristiky nejbližšího souseda i heuristiky vytvořené pomocí velkého jazykového modelu ve všech variantách, přičemž nejlépe dopadly varianty s časovými okny a s elektromobily. Vygenerované heuristiky překonali i Clarke-Wrightův algoritmus úspor ve třech variantách, čímž potvrdili účinnost tohoto přístupu, i když srovnání se známými optimálními řešeními ukazuje, že stále existuje značný prostor pro další zlepšování.

Keywords

Genetic Programming, Large Language Models, Vehicle Routing Problem, Multi-Depot VRP, VRP with Time Windows, Green VRP, Electric VRP, Heuristic Evolution, Heuristics, Hyper-Heuristics, Evolutionary Algorithms

Klíčová slova

Genetické programování, velké jazykové modely, problém směrování vozidel, Multi-Depot VRP, VRP s časovými okny, zelené VRP, elektrické VRP, evoluce heuristik, heuristiky, hyperheuristiky, evoluční algoritmy

Reference

ČERVENKA, Rostislav. *Automated Design of Constructive Heuristics for the Vehicle Routing Problem*. Brno, 2026. Master's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor Ing. David Sedlák

Rozšířený abstrakt

Efektivní logistika a plánování tras jsou jedny z ekonomicky nejvýznamnějších optimalizačních problémů, kterým moderní průmysl čelí. Ať už se jedná o doručování balíků, doplňování zásob v supermarketech nebo svoz odpadu, efektivní plánování tras má přímý dopad na provozní náklady, ekologickou stopu a spolehlivost služeb. Základem těchto procesů je problém směřování vozidel (VRP). Tento kombinatorický problém se zabývá nalezením tras pro skupinu vozidel tak, aby byla obsloužena skupina zákazníků za použití minimálních nákladů. Tento problém je NP-těžký, což znamená, že nalezení optimálního řešení pro instance velkého rozsahu je často výpočetně neproveditelné. V praxi se proto používají heuristiky, které umožňují generovat kvalitní řešení za přijatelnou dobu.

Klasické heuristiky, jako je algoritmus nejbližšího souseda (NN), budují trasy iterativním vkládáním zákazníků na základě pevně daných, ručně navržených pravidel. Tyto přístupy jsou sice účinné u jednoduchých případů, ale jejich rozhodovací logika není snadno přenositelná na složitější nebo strukturně odlišné varianty problému. V reálném nasazení se VRP zřídka vyskytuje ve své základní podobě. Praktické faktory, jako jsou striktní časové požadavky, omezené kapacity, jsou jen některé z možných variant omezení, přičemž každé z nich přináší další úroveň složitosti. Manuální návrh rozhodovací logiky pro každou novou variantu je časově náročný a nespolehlivý.

Tato práce zkoumá využití hyperheuristického přístupu založeného na genetickém programování (GPHH) k automatizaci tohoto návrhu. Namísto ručního vytváření pravidel využívá GPHH evoluci k objevování funkcí, které řídí stavbu trasy, a vyvíjí tak logiku přizpůsobenou konkrétnímu problému. Navrhl a implementoval jsem systém, který vyvíjí prioritní funkce pro šestnáct variant problému kombinujících omezenou kapacitu, časová okna, vícero dep a omezení dojezdu elektromobilů. Tato zvolená evoluční metoda byla aplikována jednotně na všechny varianty bez nutnosti dodatečných úprav při změně použitých omezení.

Pro vyhodnocení byly vyvinuté heuristiky porovnány s Clarke-Wrightovým algoritmem úspor (CW), algoritmem NN a heuristikami generovanými velkým jazykovým modelem (LLM). Při použití LLM byl model vyzván k vytvoření prioritní funkce strukturně podobné funkcím z genetického programování, avšak bez jakékoli zpětné vazby založené na fitness. Toto srovnání poskytuje pohled na to, jak si moderní modely vedou v porovnání se specializovaným evolučním vyhledáváním v roli nástrojů pro tvorbu heuristik.

Výsledky potvrzují, že GPHH se osvědčil jako velmi účinný přístup k automatizovanému návrhu heuristik. Vyvinuté heuristiky překonaly jak základní algoritmus NN tak i metodu založenou na LLM ve všech šestnácti testovaných variantách. Nejvýraznější zlepšení bylo zaznamenáno u variant s časovými okny a s omezeními dojezdů, kde lokálně orientované metody nedokážou předvídat budoucí omezení, která nastanou během tvorby trasy. Například u varianty VRPTW bez kapacitních omezení snížilo genetické programování celkové náklady o téměř 34 % oproti NN. U tří variant, konkrétně s elektromobily s omezeným dojezdem, předčily vyvinuté heuristiky i výsledky CW. Srovnání se známými optimálními řešeními odhalilo nedostatky v rozmezí 23 % až 53 %, což ukazuje, že navzdory úspěšné generalizaci napříč různými variantami stále existuje výrazný prostor pro zlepšení celkové kvality řešení.

Závěry ukazují, že GPHH představuje praktickou a flexibilní alternativu k manuální tvorbě heuristik, která se stává tím výhodnější, čím více roste složitost problému. Vzhledem k silným výsledkům algoritmu CW se jako slibný směr budoucího výzkumu jeví využití genetického programování v selektivních konstruktivních metodách, které by mohly efektivně spojit vysokou kvalitu klasických algoritmů s flexibilitou evolučního vyhledávání.

Automated Design of Constructive Heuristics for the Vehicle Routing Problem

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of Ing. David Sedlák. I have listed all the literary sources, publications, and other sources that were used during the preparation of this thesis. I have used Grammarly to correct spelling and other language mistakes. I used GitHub Copilot when working on the software.

.....
Rostislav Červenka
May 18, 2026

Acknowledgements

I would like to express my gratitude to my supervisor, Ing. David Sedlák, for his support while working on this thesis.

Contents

1	Introduction	6
2	Vehicle Routing Problem	8
2.1	Introduction and Background	8
2.2	Formal Definition	9
2.3	Variants	9
2.4	Solution Methods	13
3	Hyper-heuristics	16
3.1	Introduction	16
3.2	Classification	16
3.3	Genetic Programming	18
3.4	Solving the Vehicle Routing Problem using Genetic Programming	22
4	Design and Implementation	24
4.1	System architecture	24
4.2	Chosen Vehicle Routing Problem Variants and Datasets	28
4.3	Variant combinations	30
4.4	Genetic Programming Terminals and Functions	31
4.5	Evolutionary Process Implementation	33
4.6	Route Construction	34
4.7	Baseline Heuristics Implementation	40
5	Experimentations and Results	42
5.1	Experimental Setup	42
5.2	Hyperparameter Analysis	44
5.3	Terminal Set Analysis	52
5.4	GP Heuristic Performance	55
5.5	Heuristic Generation using Large Language Models	58
5.6	Final Evaluation	63
6	Conclusion	67
	Bibliography	68
A	LLM Generation Prompt	71
B	Examples of evolved trees	73

List of Figures

2.1	Vehicle Routing Problem algorithm classification. Taken from [16, 18]	14
3.1	GP Syntax Tree representing the expression $((a + (-1)) - c) \times (d \div e)$.	19
3.2	An example of crossover in GP, taken from [6].	21
4.1	Modular System Architecture	25
5.1	Impact of population and generation sizes on mean % improvement over NN.	44
5.2	Convergence curves for each population/generation configuration pooled across all variants.	45
5.3	Mean improvement over NN categorized by problem constraints.	45
5.4	Impact of crossover and mutation rates on mean % improvement over NN.	46
5.5	Impact of tournament size on mean % improvement over NN.	47
5.6	Impact of initialization and mutation depth on mean % improvement over NN.	48
5.7	Tree size evolution (nodes) for Mutation Depth 0–1.	49
5.8	Tree size evolution (nodes) for Mutation Depth 0–3.	49
5.9	Tree size evolution (nodes) for Mutation Depth 1–5.	50
5.10	Correlation between the tree size of best-performing individuals (per variant $\times$ run) and percentage improvement over NN.	50
5.11	Impact of training set size on mean % improvement over NN.	51
5.12	Frequency of terminal usage within the best-performing individuals	52
5.13	Impact of different terminal sets rates on mean % improvement over NN.	54
5.14	Mean percentage improvement over NN categorized by problem constraint.	55
5.15	Terminal utilization counts across different set configurations.	56
5.16	Mean best-per-generation fitness across five runs for the uncapacitated Vehicle Routing Problem with Time Windows (VRPTW) (shaded: ± 1 std).	57
5.17	Tree structure of the best individual for the uncapacitated GVRP-MD.	57
5.18	Distribution of heuristic sizes for all valid LLM candidates.	61
5.19	Frequency of Terminal Usage in LLM answers	62
5.20	Normalized performance comparison of all heuristics	65
B.1	Tree structure of the best-performing individual for uncapacitated VRP.	73
B.2	Tree structure of the best-performing individual for uncapacitated MDVRPTW.	74
B.3	Tree structure of the best-performing individual for capacitated MDGVRP.	74

List of Tables

2.1	Comparison of Classic VRP and Green VRP Constraints.	11
4.1	Attributes of the Internal VRP Instance Object	26
4.2	Problem Variant Constraint Mapping	31
4.3	Heuristic Performance Comparison: Nearest Neighbor vs. Savings	40
5.1	Baseline GP Hyperparameters	42
5.2	Terminal set configurations across different pruning levels.	53
5.3	Performance Comparison: Aggregated Best GP Results vs. NN	56
5.4	Comparative Size Statistics for LLM-Generated Heuristics	60
5.5	Performance Comparison: LLM vs. NN Baseline	63
5.6	Performance Comparison: GP, LLM, NN, and CW Savings	64
5.7	GP Performance Against Optimal Solutions	66

List of abbreviations

API	Application Programming Interface.
AST	Abstract Syntax Tree.
CVRP	Capacitated Vehicle Routing Problem.
CW	Clarke and Wright Savings Algorithm.
DEAP	Distributed Evolutionary Algorithms in Python.
DVRP	Dynamic Vehicle Routing Problem.
EV	Electric Vehicle.
G-VRPTW	Green Vehicle Routing Problem with Time Windows.
G-VRPTW-MD	Green Vehicle Routing Problem with Time Windows and Multiple Depots.
GP	Genetic Programming.
GPHH	Genetic Programming Hyper-Heuristic.
GVRP	Green Vehicle Routing Problem.
GVRP-MD	Green Vehicle Routing Problem with Multiple Depots.
HFVRP	Heterogeneous Fleet Vehicle Routing Problem.
HH	Hyper-Heuristic.
IQR	interquartile range.
LLM	Large Language Model.
MDVRP	Multiple Depot Vehicle Routing Problem.
MDVRPTW	Multi-Depot Vehicle Routing Problem with Time Windows.
MILP	Mixed-Integer Linear Program.
MoE	Mixture-of-Experts.
MTVRP	Multi-Trip Vehicle Routing Problem.
NN	Nearest Neighbor.
OVRP	Open Vehicle Routing Problem.
RGS	Route Generation Scheme.

RP	Routing Policy.
SDVRP	Split Delivery Vehicle Routing Problem.
SVRP	Stochastic Vehicle Routing Problem.
TDVRP	Time-Dependent Vehicle Routing Problem.
TSP	Traveling Salesman Problem.
TSPLIB	Traveling Salesman Problem Library.
VRP	Vehicle Routing Problem.
VRPPD	Vehicle Routing Problem with Pickup and Delivery.
VRPTW	Vehicle Routing Problem with Time Windows.
XML	Extensible Markup Language.

Chapter 1

Introduction

Efficient logistics and transportation planning are some of the most economically significant optimization problems faced by modern industry. Whether it involves package delivery, supermarket restocking, waste transport, or passenger transport, constructing optimal routes quickly and efficiently has a significant impact on operational costs, fuel consumption, and service reliability. At the core of this lies the Vehicle Routing Problem (VRP), a combinatorial optimization problem that aims to find a set of routes for a fleet of vehicles to service a group of customers at minimal cost. Since its introduction, the VRP has become one of the most studied problems in combinatorial optimization, reflecting both its complexity and widespread application.

The VRP is NP-hard, meaning that finding the optimal solution for large-scale instances is not computationally feasible. In practice, constructive heuristics are commonly used to generate high-quality solutions within reasonable time limits. These methods create routes by iteratively inserting customers into the solution based on a predetermined set of manually designed decision rules, with the Nearest Neighbor (NN) algorithm being a good example. While effective for the problem settings they were designed for, hand-crafted heuristics have one major drawback: their decision-making process cannot be readily applied to more complex variants of the same problem.

In real-world applications, the VRP usually does not appear in its most basic form. Practical constraints, such as vehicle capacity limits, customer time windows, multi-depot locations, and environmental factors, including vehicle range limits, lead to a large set of problem variants, each of which introduces additional complexity. Manually redesigning routing logic for each new variant is both time-consuming and unreliable, as rules that work well in one setting frequently perform poorly in another.

To address this challenge, this thesis explores the use of Genetic Programming Hyper-Heuristic (GPHH) to automate the design of constructive heuristics for the VRP and its variants. Instead of manually creating routing rules, GPHH uses evolutionary search to automatically discover scoring functions that guide route construction, evolving logic specifically to the given problem. This approach provides greater flexibility and reusability across variant types, as the same evolutionary framework can be applied to any problem, where a set of routing features and a fitness metric can be defined.

Recently, the development of Large Language Models (LLMs) has prompted discussion of their potential contributions to heuristic development. LLMs have demonstrated the ability to generate functional code for a wide range of programming tasks, suggesting that a routing heuristic could be generated by prompting them. This thesis will therefore also evaluate LLM-generated heuristics alongside the Genetic Programming (GP)-evolved

and classical approaches, comparing all three approaches across different VRP variants of varying complexity.

The remaining sections of this thesis are structured as follows. In Chapter 2, the VRP is introduced, along with its definition, background, the problem variants considered in this thesis, and with an overview of existing solution methods. The theory behind Hyper-Heuristics (HHs) and GP is described in Chapter 3 together with their applications in VRPs. Chapter 4 describes the design and implementation of the system developed for this thesis. Chapter 5 presents the experimental work and concludes with a comparative evaluation of all examined heuristic approaches.

Chapter 2

Vehicle Routing Problem

2.1 Introduction and Background

As defined by Toth and Vigo [29], VRP is a combinatorial optimization problem that aims to find the optimal set of routes for a fleet of vehicles based at a central depot to service a set of distributed customers. When the fleet size is restricted to one, the VRP's structure becomes identical to that of the Traveling Salesman Problem (TSP) [1].

When evaluating a solution to the VRP, multiple criteria must be considered, in addition to the basic distance traveled by all vehicles. These include execution time, the number of vehicles used, as well as flexibility and robustness [26].

2.1.1 Complexity

Since the VRP generalizes the Traveling Salesperson Problem, it is classified as **NP-hard**. This means that there is no known algorithm capable of solving the problem optimally in polynomial time, $O(n^k)$, where n is the number of customers and k is a constant [17].

As the number of customers in the problem increases, the search space grows factorially, making exact methods computationally prohibitive for large-scale instances. While these methods guarantee optimality, their practical application is generally limited to problem sizes of 50 to 100 nodes [30, 14]. Even state-of-the-art implementations, such as Branch-Price-and-Cut, are restricted to a few hundred customers [22]. For larger problems, approximate approaches become necessary.

Konstantakopoulos et al. [15] categorize these into heuristics, comprising construction, two-phase, improvement methods, and metaheuristics, which employ either population-based search (e.g., evolutionary algorithms) or local search. The complete classification is illustrated in Figure 2.1

2.1.2 History

The VRP has been studied extensively since 1959, when it was first introduced by American scientists G. B. Dantzig and J. H. Ramser in their article 'The truck dispatching problem' [10]. In this work, the authors described a scenario in which trucks delivered oil to a number of gas stations.

The problem acquired its modern name five years later (1964), when Clarke and Wright gave it its current form with their paper: 'Scheduling of Vehicles from a Central Depot to a number of Delivery Points' [8, 5].

2.1.3 Real-World Applications

The practical relevance of the VRP cannot be overstated, as transportation represents a significant component of the global economy. In many industries, distribution costs account for 10% to 20% of the total value of goods sold [29]. Therefore, even small improvements in routing efficiency can turn into substantial financial savings and competitive advantages.

The primary applications of the VRP are in logistics and supply chain optimization. Organizations specialized in the distribution of physical goods, including large-scale retail networks, e-commerce platforms, and courier services, employ VRP-based modeling as a core methodology for transportation planning. By mathematically optimizing transport trajectories, these systems minimize fuel consumption and labor costs while simultaneously increasing the capacity throughput of the existing logistics infrastructure.

Another important application area is urban transportation and mobility services. Ride-sharing platforms and taxi services utilize VRP formulations to dynamically assign vehicles to customers. It is also widely used in waste collection and other such services. Route optimization is used to plan efficient routes for garbage collection, street cleaning, and snow removal [29].

2.2 Formal Definition

The VRP is generally defined on a directed graph $G = (V, E, C)$. The set of vertices is denoted by $V = \{v_0, v_1, \dots, v_n\}$, where v_0 represents the *depot* and the set $V' = V \setminus \{v_0\}$ represents the n customers (or requests) to be serviced.

The set of arcs E connects the vertices and is defined as:

$$E = \{(v_i, v_j) \mid v_i, v_j \in V, i \neq j\} \quad (2.1)$$

Associated with E is a cost matrix $C = (c_{ij})$, where c_{ij} represents the travel cost (distance or time) associated with the arc $(v_i, v_j) \in E$.

The objective of the VRP is to determine a set of routes for a fleet of K identical vehicles based at the depot v_0 , such that the following conditions are met [23]:

1. Each vertex $v_i \in V'$ is visited exactly once.
2. All routes start and end at the depot v_0 .
3. The total routing cost, as defined by C , is minimized.

2.3 Variants

The basic VRP formulation is insufficient for today's challenges and the practical requirements of modern logistics, leading to the creation of a large number of variants and their combinations. In the following subsections, we will present the most common variants and the ones most relevant to this thesis.

2.3.1 Capacitated Vehicle Routing Problem

The Capacitated Vehicle Routing Problem (CVRP) is the fundamental variant of the VRP, appearing as a core component of many other variants. It offers a closer approximation of real-world operations than the unconstrained VRP.

In this model, the fleet consists of K homogeneous vehicles, each with the same carrying capacity Q . Every customer i is assigned a positive demand value d_i . The demand can represent the weight, volume, or quantity of the delivered package. The defining constraint of this variant is that the sum of demands from all customers assigned to a specific route must not exceed the vehicle’s capacity, as can be seen in equation 2.2

$$\sum_{i \in \text{Route}_k} d_i \leq Q \quad \forall k \in \{1, \dots, K\} \quad (2.2)$$

To expand on the CVRP and model the real world more accurately, the Heterogeneous Fleet Vehicle Routing Problem (HFVRP), or the Mixed Fleet VRP, was developed, in which the fleet consists of vehicles with different carrying capacities and costs (e.g., cost representing the amount of fuel consumed per kilometer) [19].

2.3.2 Vehicle Routing Problem with Time Windows

The VRPTW involves assigning each client a specific time window within which they must be served.

For every customer i , a time window $[e_i, l_i]$ is defined. This window consists of the earliest arrival time e_i , when a service can begin, and the latest arrival time l_i , by which the service must start. Additionally, a service time s_i is included to represent the time required to process the delivery, including tasks such as unloading and administrative requirements.

A key characteristic of the VRPTW is how it handles early arrivals. If a vehicle arrives at customer i before time e_i , it is allowed to wait, however the service cannot commence until e_i [13].

In the literature, this problem is further categorized based on the strictness of the constraints:

- **Hard Time Windows:** The vehicle must arrive before l_i . Missing the deadline makes the solution infeasible. This is the standard definition used in most benchmarks (e.g., Solomon’s instances [28]).
- **Soft Time Windows:** The vehicle can arrive after l_i , but a penalty cost is added to the objective function. This model’s scenarios where lateness is undesirable but tolerable (e.g., a package arriving 5 minutes late).

In this variant, the most crucial factor is time management. Consequently, it is quite common not to try to minimize the total distance traveled, but rather the total time taken by the whole fleet [19].

2.3.3 Green Vehicle Routing Problem

The Green Vehicle Routing Problem (GVRP) (often synonymous with the Electric VRP or Alternative Fuel VRP) extends the classic VRP by incorporating constraints related to limited vehicle range and the availability of refueling infrastructure. While the classic VRP assumes vehicles have sufficient fuel to complete any feasible route, the GVRP requires the algorithm to actively manage energy consumption and insert intermediate stops.

Key Structural Differences

To transition from a classical VRP to a GVRP, three fundamental components of the problem must be redefined: the network graph, the vehicle state, and the temporal constraints.

Table 2.1: Comparison of Classic VRP and Green VRP Constraints.

Feature	Classic VRP	Green VRP
Graph Nodes	Customers + Depot	Customers + Depot + Stations (S)
Node Visits	Exactly once (Customers)	Multiple/Zero (Stations)
Constraint	Cargo Capacity	Cargo + Energy Range (Q)
Time Factor	Travel + Service	Travel + Service + Refueling (r_k)

Augmented Graph Structure The vertex set is expanded to include a set of Refueling Stations S . Unlike customer nodes, station nodes are not mandatory, they may be visited multiple times by different vehicles or skipped entirely. These stations are typically modeled with unlimited capacity, allowing multiple vehicles to refuel simultaneously.

The Range Constraint In addition to cargo capacity, the GVRP tracks energy range. Each vehicle is assigned a Maximum Driving Distance Q . Traveling from i to j consumes a distance c_{ij} ; whenever a vehicle visits a station $k \in S$, this accumulated consumption is reset to zero, extending the vehicle’s operational range beyond the initial limit Q .

Time Calculation and Refueling Latency The total route duration T is extended by a refueling component r_k . While travel time t_{ij} remains proportional to distance ($t_{ij} = c_{ij}/v$), the overall schedule must account for the time spent at each station visit [19].

Refueling and Charging Models

There are many different ways refueling time is modeled in literature, the most common variants include [19]:

1. **Fixed Refueling Time:** Time spent at a station is constant, regardless of the energy level. This model’s battery-swapping stations or gas-station stops.
2. **Linear Charging 2.3** Refueling time is linearly proportional to the energy replenished, see Equation 2.3

$$t_{\text{refuel}} = \alpha \times (Q_{\text{target}} - Q_{\text{current}}) \quad (2.3)$$

where α represents the constant charging rate.

3. **Non-Linear Charging:** A more realistic model for Electric Vehicles (EVs), recognizing that battery charging slows down significantly as it approaches 100%.
4. **Recharge Policy (Full vs. Partial):** Specifies whether the vehicle is forced to restore full capacity at every stop (Full) or is permitted to get just enough energy to reach the next destination (Partial).
5. **Heterogeneous Stations:** Stations may have different charging speeds, the refueling time depends not only on the energy required but also on the specific technology available at the chosen station node.

Hybrid Variants

Beyond the single-technology model, two hybrid extensions are prominent in recent research. The **Hybrid Fleet VRP** optimizes a mixed composition of electric and internal combustion vehicles. Conversely, the **Hybrid Vehicle VRP** focuses on vehicles equipped with dual propulsion systems. The latter introduces a significant optimization challenge: the algorithm must dynamically decide when to switch between available power sources to maximize efficiency across varying route segments [13].

2.3.4 Multiple Depot Vehicle Routing Problem

In large-scale logistic networks, serving all customers from a single central facility is often impractical or inefficient. The Multiple Depot Vehicle Routing Problem (MDVRP) addresses this by introducing a set of multiple depots, $D = \{d_1, \dots, d_m\}$, distributed across a given area.

The introduction of multiple depots adds an assignment phase to the routing process. The solver must address two simultaneous decisions:

1. **Allocation:** Determining which depot will service which customer. This is typically based on proximity or clustering algorithms.
2. **Routing:** Constructing optimal routes for the vehicles assigned to each specific depot.

Each vehicle must return to its starting depot at the end of its route, ensuring the fleet does not cluster at a single location and remains ready for the next planning period. [13, 19].

2.3.5 Vehicle Routing Problem with Pickup and Delivery

The Vehicle Routing Problem with Pickup and Delivery (VRPPD) changes the nature of the routing problem. In VRPPD, the depot serves only as a storage facility for vehicles, while goods are exchanged between customer pairs.

Each request i is defined by a pair of locations: a Pickup Point (P_i) and a Delivery Point (D_i). Two new constraints are introduced:

1. **Ordering:** The pickup location P_i must be visited before the delivery location D_i .
2. **Coupling:** Both P_i and D_i must be served by the same vehicle.

This variant is widely used in on-demand transport services, like ride-sharing or Reverse Logistics (collecting returns/waste) [13, 19].

2.3.6 Other Notable Variants

Beyond the formulations core to this thesis discussed above, there exist many other variants, here are the most interesting ones [15]:

- **Dynamic Vehicle Routing Problem (DVRP):** A real-time variation where input data (such as new customer requests or traffic delays) is revealed dynamically while the route is being executed, requiring on-the-fly re-optimization [26, 25].

- **Stochastic Vehicle Routing Problem (SVRP):** Instead of fixed values, at least one parameter (e.g., customer demand, travel time) is modeled as a random variable with a known probability distribution, requiring solutions that are robust to uncertainty [29].
- **Open Vehicle Routing Problem (OVRP):** Vehicles are not required to return to the depot after visiting the last customer, modeling scenarios where drivers use their own vehicles or finish at their homes [29].
- **Multi-Trip Vehicle Routing Problem (MTVRP):** Vehicles can return to the depot to reload and perform multiple distinct routes within a single planning horizon, effectively increasing the fleet’s total capacity [29].
- **Time-Dependent Vehicle Routing Problem (TDVRP):** The travel time between two nodes is not constant but fluctuates based on the time of day, specifically modeling real-world traffic problems [25].
- **2D and 3D VRP (VRP with Loading Constraints):** Integrates the Bin Packing Problem into the routing logic, ensuring that the delivered packages not only fit by total weight but also physically inside the vehicle’s cargo space dimensions [29].
- **Split Delivery Vehicle Routing Problem (SDVRP):** Relaxes the constraint that a customer must be visited exactly once, allowing a single customer’s demand to be fulfilled by multiple vehicles, allowing for orders too large for one vehicle [29].

2.4 Solution Methods

As illustrated in Figure 2.1, standard classifications divide VRP algorithms into three primary categories: Exact Methods, Heuristics, and Metaheuristics.

2.4.1 Exact methods

Finding a global optimum is the central theme of Exact Methods. These techniques exhaustively explore the solution space by either fully scanning it or by applying high-level mathematical programming. While they guarantee an optimal result, they frequently run into a „computational wall“ because the VRP is fundamentally an NP-hard problem. This means their use is mostly restricted to smaller benchmarks, typically with fewer than 100 customers, where the massive processing overhead hasn’t yet become a bottleneck.

Most of these strategies start by framing the task as a Mixed-Integer Linear Program (MILP) and using tree-search tools such as Branch-and-Bound or Branch-and-Cut to narrow down the best possible route. These methods systematically calculate lower bounds to „prune“ irrelevant parts of the search space, avoiding a full brute-force exploration while still guaranteeing optimality [4].

2.4.2 Heuristics

Given the limitations of exact methods, other approaches had to be considered. One of them is classical heuristics, which trade off solution optimality for speed, allowing them to solve large-scale instances. Heuristics for VRP can be divided into three classes [4]:

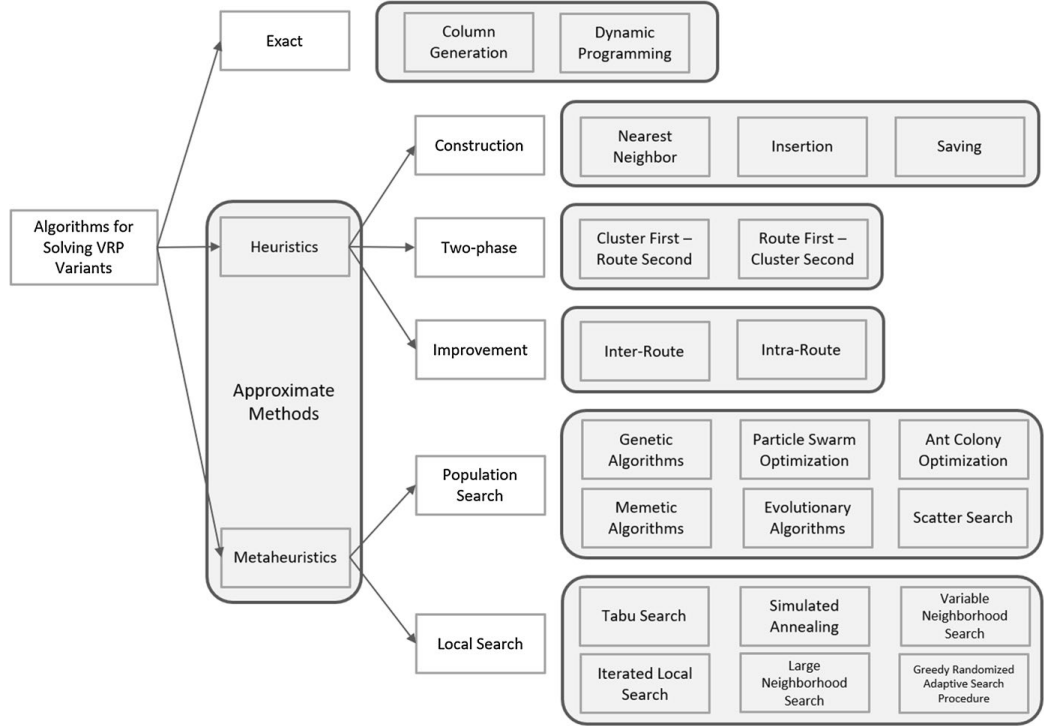


Figure 2.1: Vehicle Routing Problem algorithm classification. Taken from [16, 18]

- **Constructive Methods:** These types of algorithms work by continuously adding one node at a time until all customers are served. They are fast but often produce suboptimal routes.
- **Two-Phase Methods:** This category breaks down the problem into two simpler sub-problems: Clustering: assigning customers to vehicles, and Routing: ordering the visits.
- **Improvement Methods:** Also known as local search heuristics, they take an existing complete solution, which is often generated by a constructive heuristic, and iteratively modify it to improve its quality.

Nearest Neighbor

The NN algorithm is one of the simplest constructive heuristics. It employs a greedy strategy where the vehicle starts at the depot and, at each step, visits the closest unvisited customer. This process repeats until the vehicle's capacity is reached or all customers have been served. While extremely fast ($O(n^2)$), it often leaves isolated, distant customers at the end of the route, resulting in very high final route costs.

Savings Algorithm

The Clarke and Wright Savings Algorithm (CW) is a constructive method that generally outperforms the NN algorithm.

It begins with an initial solution in which every customer is served by a dedicated vehicle, yielding n routes. The algorithm then calculates the „savings“ from merging two

routes, linking customers i and j directly, without returning to the depot. The merges with the highest positive savings are performed iteratively, provided they satisfy constraints, until no further improvements are possible.

2.4.3 Metaheuristics

Metaheuristics improve upon classical heuristics by enabling the search to escape local optima. They can be classified into [4]:

- **Local Search:** Improving a single route iteratively (e.g., Tabu Search, Simulated Annealing).
- **Population Search:** Evolving a set of solutions (e.g., Genetic Algorithms, Ant Colony Optimization).

Chapter 3

Hyper-heuristics

3.1 Introduction

Standard metaheuristics often require extensive manual calibration for each new problem instance, as they operate directly in the solution space, searching for the best set of routes. HHs aim to raise the level of generality by shifting the search to the heuristic space, looking not for a solution but for an algorithm capable of producing them.

The motivation behind HHs is to automate the design of heuristic methods. HHs aim to produce algorithms that are not only effective but also reusable across different problem instances. This is also why HHs have been applied to many problems, such as the VRP, the TSP, the Nurse Rostering Problem, various packaging problems, and more [20].

3.1.1 A Formal Definition

“A HH can be defined as a search algorithm for solving an optimization problem P , whose decision variables are heuristics, rather than direct solution variables in the optimization problem p under consideration. To solve P , HH explores at a higher level a heuristic space H of heuristic configurations h , which at a lower level generate direct solutions s in the solution space S for problem p . Two search spaces can thus be defined, namely a heuristic space H of P and a solution space S of p , each associated with an objective function, within the two-level framework.” [20]

3.2 Classification

HHs are categorized by the nature of the underlying operators and the source of the heuristic logic. Based on the former, constructive heuristics generate a feasible solution by iteratively adding elements to an initially empty set, while perturbative heuristics (or improvement heuristics) operate on a complete initial solution, applying local transformations to explore its neighborhood in search of a higher-quality result. Regarding the latter, selection methods choose from a predefined set of low-level heuristics, while generation methods construct new heuristics from smaller components. Based on this, HHs can be classified into four categories: selection-constructive, selection-perturbative, generation-constructive, and generation-perturbative [20].

Selection constructive methods choose a low-level heuristic to apply at each step of solution construction. Selection techniques include case-based reasoning, local search, population-based, adaptive, and hybrid methods.

Algorithm 1 Selection constructive HH algorithm [20]

```

1: procedure SELECTIONCONSTRUCTIVEHYPERHEURISTIC( $p, L$ )
2:   Initialize solution  $s$  to be empty
3:   repeat
4:     Use technique  $T$  to select a low-level construction heuristic  $L_i$  from  $L$ 
5:     Apply  $L_i$  to extend the solution  $s$ 
6:   until solution  $s$  is completely constructed
7:   return  $s$ 
8: end procedure

```

Selection perturbative HHs select which perturbative heuristics to apply at each step to improve an initial solution. These methods use either a single-point or a multipoint search to select the appropriate low-level perturbative heuristic. In single-point search, the system requires distinct components for heuristic selection and move acceptance. Multipoint search uses population-based methods (like evolutionary algorithms), which inherently handle both selection and acceptance.

Algorithm 2 Selection perturbative HH algorithm – single-point search [20]

```

1: procedure SELECTIONPERTURBATIVEHYPERHEURISTIC( $p, L$ )
2:   Create an initial solution  $s_0$  using a random or constructive heuristic
3:   repeat
4:     Use the heuristic selection technique  $h$  to select a perturbative heuristic  $L_i$  from  $L$ 
5:     Apply  $L_i$  to solution  $s_i$  to produce the perturbed solution  $s_{i+1}$ 
6:     Use the move acceptance technique  $M$  to accept the move or not
7:     If the move is accepted  $s_i \leftarrow s_{i+1}$ 
8:   until the termination criterion
9:   return  $s_i$ 
10: end procedure

```

Generation constructive methods create new low-level construction heuristics for a problem domain. The generated heuristic builds an initial solution, which is then further optimized. GP is usually used to evolve these heuristics by combining existing heuristic components and problem characteristics via arithmetic and conditional operators. The resulting heuristics can be disposable, created for a specific problem, or reusable, used for unseen instances.

Generation perturbative HHs create new perturbative heuristics by combining existing low-level heuristics and acceptance criteria. This is usually achieved through conditional logic (if-then-else statements) [20].

3.2.1 Offline vs Online learning

Another way HHs can be classified is based on when their learning process occurs.

In the Online version, learning takes place as the algorithm solves an instance of the problem. This approach usually produces higher-quality results, but because it is tailored

to a single instance, we lose the ability to generalize or reuse the learned strategy on new, unseen problem instances.

Offline learning separates the process into distinct training and testing phases. The goal of the training phase is to discover the best generic, reusable heuristic using a representative set of training instances. This learned heuristic is then fixed and evaluated on new instances during the testing phase. [20].

3.3 Genetic Programming

GP is an Evolutionary Algorithm based on the principles of biological evolution and natural selection. What distinguishes it from classical optimization techniques is its ability to automate: instead of following hand-crafted rules, GP evolves a number of candidate computer programs, guiding them toward a solution that suits the specific problem being addressed.

The process simulates natural selection by maintaining a large pool of candidate solutions, known as the population. The size of it, referred to as the population size, is a critical parameter that determines the amount of diversity available to the search. The algorithm proceeds iteratively through a series of steps called the generations [24].

In each generation, every program in the population is evaluated to determine its fitness, a measure of how well it solves the target problem. The best-performing individuals are selected to become parents. These parents create the next generation of programs using genetic operators that simulate natural reproduction. By repeating this process for a pre-defined number of generations, the system evolves from simple attempts into sophisticated heuristics [20]. This process is properly outlined in Algorithm 3.

Algorithm 3 Genetic Programming Algorithm [20]

- 1: Initialize the population with random programs
 - 2: Evaluate the fitness of each individual in the initial population
 - 3: **repeat**
 - 4: Select parents from the current population
 - 5: Apply genetic operators (crossover and mutation) to create offspring
 - 6: Update the population with offspring
 - 7: Evaluate the fitness of the new population
 - 8: **until** Termination criteria are met
 - 9: **return** The best solution found during the search
-

3.3.1 Representation

While there are multiple ways to represent individuals in GP, the most common approach, and the one used in this thesis, is the tree-based representation, specifically Abstract Syntax Trees (ASTs) (see Figure 3.1). In this structure, the leaves of the tree are known as terminals, while the internal nodes are called functions.

The terminal set is composed of constants and variables. The constants are typically generic numerical values, whereas the variables are specific to the problem domain (e.g., *distance*, *time*, *capacity*).

The function set comprises the arithmetic and logical operators used to combine these terminals. Common operators include $+$, $-$, $\times$, $/$, $\max$, and $\min$. The number of children

associated with each internal node is determined by the function's arity (e.g., $+$ requires two children, while $\sin$ requires only one).

A critical requirement in GP is the closure property, which ensures that the evolved programs are always executable and never crash. A common issue arises with the division operator $/$, which fails if the denominator is zero. To prevent this, standard GP implementations replace it with protected division. This modified operator checks the denominator before execution and returns a safe default value (typically 0 or 1) if the denominator is zero, ensuring the function never fails [24].

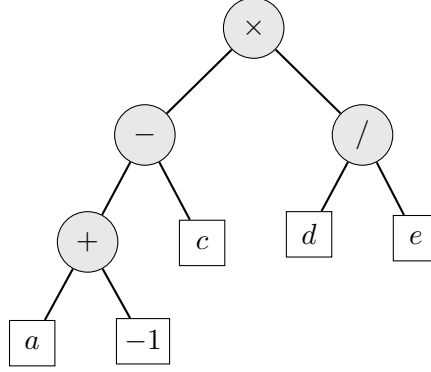


Figure 3.1: GP Syntax Tree representing the expression $((a + (-1)) - c) \times (d \div e)$.

3.3.2 Population Initialization

To begin with GP, we must first create an initial population. This population is randomly generated using three main methods described in [24]: the full and grow methods, and their combination, the Ramped Half-and-Half method.

The Full Method generates trees where every branch reaches the maximum depth D_{max} . Nodes at depths less than D_{max} are randomly chosen from the function set, while nodes at depth D_{max} are chosen strictly from the terminal set. This results in all individuals having a similar number of nodes and tree shapes, reducing the population's diversity.

The Grow Method allows trees to have variable shapes and sizes. For any node at a depth less than D_{max} , the algorithm randomly chooses from the union of the function and terminal sets. If a terminal is chosen, that branch ends. This results in trees of varying depths, up to D_{max} .

The Ramped Half-and-Half method combines the previous two. It divides the population into groups based on depth ranges (e.g., depths 2 to D_{max}). For each depth group, half of the individuals are created using the Full method and half using the Grow method. This ensures the initial population contains a variety of different tree shapes and sizes.

3.3.3 Fitness Evaluation

Just as biological evolution relies on environmental pressure to drive natural selection, GP requires a **fitness function**. The purpose of this function is to measure the quality of candidate solutions and discover high-performing individuals.

For evolution to proceed effectively, the fitness function must provide a granular gradient of performance. It should be able to distinguish solutions that are only marginally differ-

ent. If the function lacks granularity (e.g., a binary „success/fail“), evolutionary pressure vanishes, and the algorithm becomes a random search [24].

A unique characteristic of GP is that fitness evaluation requires executing the evolved structures. Since the individuals are computer programs, they are typically evaluated by an interpreter that traverses the tree recursively.

A common issue in GP is known as *bloat*, in which programs grow unnecessarily large without improving performance. These larger syntax trees slow down evolution because they are more computationally expensive to execute. To avoid bloat, the fitness function often includes a penalty for excessive size, encouraging the algorithm to favor solutions that are both effective and compact [24].

3.3.4 Genetic Operators

Once the initial population is generated and evaluated, the evolutionary process relies on genetic operators to transform the current generation into the next. These operators are the primary search mechanism in GP, responsible for exploring the search space.

Selection

For GP to work, we must choose which individuals will serve as the starting blocks for a new generation. These individuals are selected based on their fitness value using many diverse methods, with the most common one for GP being *tournament selection*.

In **Tournament Selection**, a number of individuals are chosen at random from the population, with the fittest among them being selected as a parent. This process is repeated to secure the necessary number of parents for crossover. Crucially, this method relies on relative ranking rather than absolute fitness differences, preventing a single dominant individual from immediately swamping the population (preserving diversity) while still favoring superior programs [24].

Some other notable selection methods include *roulette with relative fitness*, where each individual has its probability to be selected calculated as $fitness/totalfitness$, and *deterministic choice*, where candidate solutions are ordered based on their fitness and the top n are picked [7].

Crossover

The crossover operator is the primary mechanism for evolution in GP. It works on the basis that combining parts of two fit individuals can lead to even better solutions, allowing an offspring to inherit beneficial traits from both parents.

The most prevalent method, and the one used in this work, is the **Subtree Crossover**. As shown in Figure 3.2, this process involves selecting a random node (crossover point) in each of the two parent trees. Then, creating an offspring by replacing the subtree started at the crossover point of the first parent with a copy of the subtree from the second parent.

Since the majority of nodes in a syntax tree are typically leaves, uniform selection of crossover points often results in the exchange of very small subtrees (e.g., swapping single terminals). To encourage structural evolution, it is standard practice to bias selection, such as the widely used „90/10 rule“, which selects internal (function) nodes 90% of the time and leaf nodes only 10% of the time [24].

Some of the other variants include *Uniform Crossover*, which flips a coin for each node in a common region to decide which parent it is taken from, *Context-Preserving*

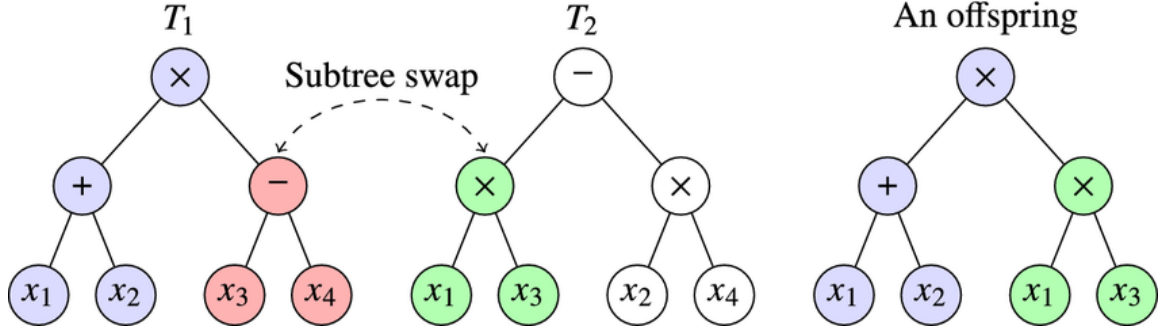


Figure 3.2: An example of crossover in GP, taken from [6].

Crossover, which aligns coordinates for crossover points, and *Size-Fair Crossover*, which selects crossover points based on the size of their subtrees [24].

Mutation

To add genetic variety to the population, mutation is used. In GP, many mutation operators are used, often simultaneously. Here are some of them [24]:

- **Subtree mutation** is the most well-known and common type. It replaces a randomly selected subtree with another randomly generated one. The generation uses methods from population initialization, with the Ramped Half-and-Half being the most common.
- **Size-fair subtree mutation** is differentiated from classic subtree mutation in that the size of the generated tree is proportional to the one being replaced.
- **Node replacement mutation** selects a random node and randomly changes it, without changing its type (terminals or functions). To keep the program executable, the replaced function nodes have to have the same arity.
- **Shrink mutation** works on the principle of subtree mutation, but the replacement is always a terminal, leading to reduced tree size.
- **Mutating constants**, which concerns only the terminals. These are either changed randomly using noise from a Gaussian distribution or systematically using various optimization tools.

Replacement Strategies

Once offspring are created using the above-mentioned genetic operators, a decision must be made about which individuals will form the population of the next generation.

The most common approach in GP is called the Generational Replacement. Using this strategy, the entire population is replaced by the newly evolved offspring at each generation. The current parents are discarded, and the offspring become the parents for the next cycle.

Because genetic operators can be and often are destructive, the best individual in the population may be lost. To prevent this, elitism can be applied alongside generational replacement. Elitism identifies the best-performing individuals from the current generation

and copies them directly into the next generation without modification. This ensures that the quality of the best solution found so far never deteriorates.

Another approach is the Steady-State Replacement, in which only some individuals are generated at a time and immediately replace the least-fit members of the current population. This approach can lead to premature convergence and a loss of diversity [24].

3.4 Solving the Vehicle Routing Problem using Genetic Programming

In this thesis, we apply GP as a generation constructive HH to solve the VRP. Unlike traditional evolutionary algorithms that evolve a direct representation of the solution (e.g., a fixed sequence of customers), this method evolves the logic used to construct the solution.

3.4.1 Routing Policies

A Routing Policy (RP) is a constructive heuristic that incrementally builds routes for the VRP. Unlike iterative improvement methods that modify complete solutions, a RP constructs a solution step by step, making local decisions to extend the current route until all customers are served.

A RP consists of two separate interacting components: a *route generation scheme* and a *priority function* [12].

3.4.2 Route Generation Scheme

The Route Generation Scheme (RGS) serves as the fixed framework of the heuristic. It defines the rules for constructing a solution, such as managing the vehicle’s capacity constraints, determining when to return to the depot, and iterating through the list of unvisited customers. This component remains static throughout the evolutionary process.

An example using Greedy Insertion:

Step 1: Initialization: Start a new route with a new vehicle from the depot.

Step 2: Feasibility: Identify all unvisited customers that can be added to the current route without violating constraints.

Step 3: Evaluation: Apply the priority function to each feasible customer to calculate their score.

Step 4: Selection: Select the candidate with the highest priority score and insert them into the route.

Step 5: Iterative Logic:

- If feasible candidates exist, go to Step 2.
- If unserved customers remain, go to Step 1 to initialize a new route.
- If all customers are served, terminate the process.

3.4.3 Priority Function

The Priority Function (often called a Scoring function or Dispatching Rule) serves as the policy’s decision-making logic. This is the component evolved by the GP system.

This function is a mathematical expression that assigns a score to a customer-vehicle pair based on the set of terminals used in GP (e.g., distance, demand, capacity). While the route generation scheme handles the mechanics of insertion, the priority function dictates which insertion is most beneficial.

The quality, or fitness, of a priority function is measured by its performance. To evaluate an individual, the function is embedded into the generation scheme to solve a set of training VRP instances. The fitness is then calculated based on the value of the resulting routes (e.g., total distance). This results in the GP evolution favoring functions that produce low-cost routes across different problem scenarios.

Chapter 4

Design and Implementation

4.1 System architecture

The primary objective of this thesis is to design and implement a software system capable of the automated design of constructive heuristics for the VRP. The system is designed to adapt to various problem constraints, generating routing policies based on the user-selected combination of VRP variants.

To achieve this objective, the system uses a modular design that clearly separates logical components, allowing for an extensible architecture. This design enables the interchangeability of VRP instances, vehicle constraints, and different features, supporting the VRP variants explored in this thesis without altering the core evolutionary logic.

The high-level interaction between these components is illustrated in Figure 4.1. The architecture consists of four main subsystems:

- Experiment Controller that manages the execution.
- Data Processing Layer (indicated in blue), responsible for parsing, storing, and augmenting data.
- Route Construction Logic (indicated in green), for evolving dispatching rules, constructing routes, and fitness evaluation.
- Analysis Module that compares results and logs performance metrics.

4.1.1 Experiment Controller

The Experiment Controller serves as the central manager of the system and as an entry point for users. This module performs three critical functions:

- **Variant Selection:** It allows the user to define the specific VRP scenario to be solved (e.g., enabling Capacity constraints, Time Windows, or multiple depots). Based on this selection, the Manager activates the relevant constraints in the system's components.
- **Configuration:** The Controller initializes the Evolutionary Logic with a standardized set of parameters.

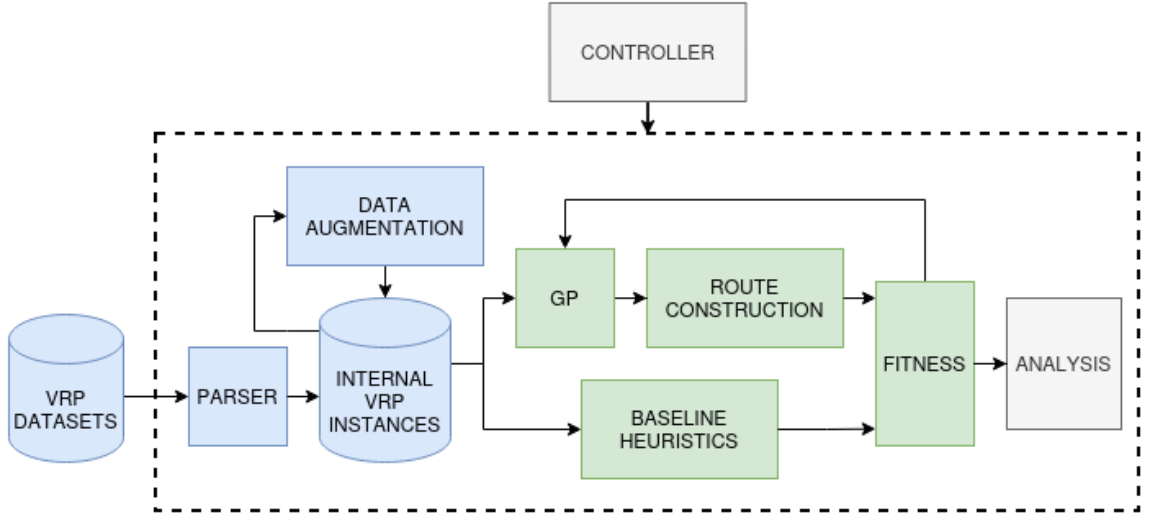


Figure 4.1: Modular System Architecture

- **Training and Testing Separation:** The Controller partitions the dataset into two separate parts. It passes the first one, the training instances, to the evolutionary loop, while the other part is used for a final validation step to ensure generalization to new, unseen data.

4.1.2 Data Processing Layer

The Data Processing Layer serves as the system’s initial component, acting as a connection point between the static benchmark files and the dynamic runtime environment. Its primary function is to transform VRP datasets into optimized memory structures that can then be interacted with by the Evolutionary Logic and Baseline Heuristics.

The process begins with the Parser component, which is designed to read and process standard VRP benchmarks. To make the system compatible with various benchmark libraries, the parser supports three different input formats: the classic Solomon text format, the standard .vrp format, and structured Extensible Markup Language (XML) files. This model also performs basic validation on the data before storing it in memory.

Once parsed, the data is converted into a standardized **Internal Instance Object**. This object serves as a standard interface for the rest of the system, abstracting away the differences between input file formats. Every instance is mapped to a set of attributes as defined in Table 4.1. To ensure consistency across variants, the parser adds neutral default values to attributes not defined in the source file, for example, assigning infinite battery capacity to variants that lack it.

Another feature of this layer is the Augmentation Module, which limits the need for variant-specific datasets. Since standard benchmarks typically exist only for specific problem types, this module dynamically generates the required training instances by modifying the constraints of the loaded data. For example, to train on a pure VRP without capacity constraints using a CVRP dataset, the module relaxes the `capacity` limit in the instance attributes. This enables the system to support all 16 variant combinations without requiring a dedicated dataset for each one.

Table 4.1: Attributes of the Internal VRP Instance Object

Attribute	Type	Description
<i>Network Structure</i>		
dimension	int	Total number of nodes in the network.
depot	ndarray (n ,)	Index identifier of the depot nodes.
coords	ndarray (n , 2)	Euclidean coordinates (x, y) for all nodes.
dist_matrix	ndarray (n , n)	Pre-computed symmetric Euclidean distance matrix.
<i>Node Attributes</i>		
demands	ndarray (n ,)	Demand quantity required by each node.
service_times	ndarray (n ,)	Duration required to service each customer.
ready_times	ndarray (n ,)	Start of the time window $[e_i]$.
due_dates	ndarray (n ,)	End of the time window $[l_i]$.
node_types	ndarray (n ,)	Classification: 0 (Depot), 1 (Customer), 2 (Station).
<i>Vehicle Constraints</i>		
capacity	float	Maximum load capacity of the vehicle.
max_travel_time	float	Maximum allowable duration for a route.
battery_capacity	float	Vehicle energy capacity

4.1.3 Evolutionary Logic

The Evolutionary Logic module is the core part of this project. Unlike traditional solvers that search for a direct solution (a set of routes), this component implements GPHH to evolve a priority function. This function acts as a decision-making rule, assigning scores to unvisited customers to guide the construction. The module is composed of two sub-components:

- **GP Core:** This component manages the population of priority functions, represented as heuristic trees. It executes the standard evolutionary cycle—selection, crossover, and mutation—to iteratively improve the quality of these functions. Its goal is to maintain population diversity and continuously discover better solutions.
- **Route Construction:** This component applies the GP-evolved heuristic rule to a VRP instance to construct a complete set of routes. By changing the validity checks in this builder, the system can generate routes for different VRP variants.

The generated routes are then passed to the Fitness evaluation and Analysis Modules to evaluate their performance, validate them on unseen test instances, and compare them with other methods.

4.1.4 Baseline Heuristics

To assess the quality of the evolved heuristics, the system includes a module for standard algorithms. These implementations provide essential performance benchmarks, establishing

both a lower bound and a competitive standard against which the GP-evolved rules are measured.

The module integrates two strategies previously defined in Section 2.4.2:

- **Nearest Neighbor (NN):** This implementation uses the same Route Construction component as the Evolutionary Logic, but without the GP part. Instead of an evolved tree, it uses a static, manually defined priority rule (minimizing distance) in the builder, ensuring a fair comparison.
- **Clarke and Wright Savings:** In contrast to the sequential construction of the NN and GP, this baseline implements a parallel merging strategy. As a result, it operates independently of the standard Route Construction module. It provides a high-quality solution to verify that the evolutionary system is competitive with established combinatorial methods.

This model’s outputs are the baseline routes for the target instances. These solutions are passed to the Fitness and Analysis modules, serving as benchmarks for evaluating the relative quality of the GP-evolved methods.

4.1.5 Fitness

The Fitness module acts as the objective function for the entire system, translating the constructed routes into a single quantitative score. This score consists of three parts. The first one, the Raw Fitness, is calculated as the total Euclidean distance of the generated routes. It is used as a performance metric for the Analysis and Basic Heuristics modules, enabling direct, fair comparisons between methods.

The second component adds a complexity penalty used exclusively when providing feedback for the GP. This penalty is added to the Raw Fitness and is proportional to the tree size of the priority function. This is done to prevent bloat and guide the evolution toward simpler, more interpretable solutions.

The third part deals with the viability of the generated solution. A significant penalty is applied when the developed priority function is unable to generate a workable route plan that serves every customer. This is done by adding four times the distance to the fitness score for every customer that is not served after all possible routes have been created. This effectively prunes out all individuals who are unable to meet the basic requirement of servicing all available customers. This is done by using a value similar to the total distance of all routes, ensuring the engine prioritizes a 100% service rate before minimizing the actual distance traveled.

4.1.6 Analysis

The Analysis module is the evaluation section of the system. It compiles the outputs from the Evolutionary Logic, Baseline Heuristics, and Fitness modules to produce performance reports. The aim of using this module is to evaluate the performance of GP and the heuristics it generates.

The key tasks performed in this module are:

- **GP Performance Monitoring:** During the course of the evolutionary run, GP statistics are monitored in real-time for each generation, including min fitness scores, averages, max fitness scores, and data concerning tree sizes.

- **Validation with Unseen Data:** To guarantee that the rules generated by the evolution process do not overfit to specific instances, this module imposes a strict separation between the data used for the evolution process. After the best-performing heuristic is obtained on the *Training Set*, this module applies it to the *Testing Set*, which consists of unseen instances. This verifies that the logic captures general routing principles rather than memorizing specific node configurations.
- **Comparative Benchmarking:** Finally, the module compares the fitness of the routes generated using the evolutionary methodology to those generated by the two benchmark methods: NN and Savings. It calculates relative improvements, expressed as percentages, between methods, demonstrating the evolutionary approach’s efficiency in solving this problem.

4.2 Chosen Vehicle Routing Problem Variants and Datasets

In this project, four different VRP variants have been used to highlight the flexibility of the proposed GPHH framework. These problem types, as defined in Section 2.3, were chosen to span a range of difficulty, from simple capacity checks to complex scheduling and refueling logic.

4.2.1 Capacitated Vehicle Routing Problem

CVRP represents the foundational routing problem, making the choice of this variant unquestionable. It provides a baseline for evaluating the core spatial logic of the GP engine.

For the fundamental CVRP experiments, the project utilizes a dataset from **Augerat et al.** [3].

- **Dataset Overview:** The chosen **Set A** contains 27 instances, ranging in size from 32 to 80 nodes. The customer distribution is Random.
- **Selection Motivation:** Set A, released in 1995, is a „gold standard“ in VRP literature and allows for comparison of the system’s performance against 30 years of established benchmarks.
- **Dataset Format:** These instances utilize the Traveling Salesman Problem Library (TSPLIB) standard. The file is organized into explicit sections: `NODE_COORD_SECTION` provides a list of node IDs and their X and Y coordinates, `DEMAND_SECTION`, which maps node IDs to specific delivery quantities. The `DEPOT_SECTION` identifies the starting node, and a global `CAPACITY` value in the header defines the vehicle limit.

4.2.2 Vehicle Routing Problem with Time Windows

VRPTW was selected to introduce strict temporal constraints that require the evolved heuristics to balance geographical proximity with customer-imposed deadlines, significantly narrowing the feasible search space.

Experiments for VRPTW use the benchmark suite established by Gehring and Homberger [11], which extends the classic Solomon [28] instances.

- **Dataset Overview:** The experiments utilize a subset of **120 instances** from the six standard problem classes: **C1**, **C2** (Clustered), **R1**, **R2** (Random), and **RC1**,

RC2 (Mixed). These instances are derived directly from the original Solomon logic but scale the problem size up to 400 customers.

- **Computational Constraints:** While the suite includes larger sets (up to 1,000 customers), this study focuses on the 100 to 400 customer range. Given the nature of GP and the high number of evaluations per generation, instances with more than 400 nodes were considered impractical for this thesis. While evaluation on larger sets is feasible, the scope was restricted to 400 nodes to maintain a consistent benchmark and ensure a comparable analysis across all 16 problem variants.
- **Selection Motivation:** By using an expansion of the Solomon set, which is widely accepted in the area of VRPTW, we maintain a baseline for structural diversity while testing the scalability of the evolved priority functions. This ensures that the learned heuristics remain robust across varying geographical distributions and time-window densities as the search space expands from the standard 100-customer benchmark toward more industrially relevant scales.
- **Dataset Format:** Instances use the **Extended Solomon** format, a column-aligned table that provides seven key metrics per node: coordinates, demand, ready time, due date, and service time. The depot defines the overall scheduling horizon via its due date.

4.2.3 Green Vehicle Routing Problem

GVRP adds the complexity of limited vehicle range and mid-route recharging logic, forcing the model to learn „look-ahead“ strategies that ensure energy feasibility without violating other constraints.

The recharging and energy management capabilities of the model are evaluated using the benchmark suite introduced by **Schneider et al.** [27].

- **Dataset Overview:** These instances are the primary standard for the Green Vehicle Routing Problem with Time Windows (G-VRPTW). It consists of 92 instances, each serving between 5 and 100 customers. It is categorized into three geographical distributions: Clustered (**C**), Random (**R**), and Random-Clustered (**RC**). Each instance includes strategically placed recharging stations (**S**) alongside customer nodes.
- **Selection Motivation:** This specific suite was selected over alternative GVRP benchmarks due to the Schneider instances integrating time-window constraints with energy limitations. It tests the heuristic’s ability to prioritize long-term route viability over short-term greedy distance gains. Choosing this dataset ensures the heuristic is tested against the most realistic and difficult version of the problem available in the literature.
- **Dataset Format:** This format expands the Solomon structure by adding Battery Capacity and fuel consumption rates to the header. Nodes are categorized by a type flag: **'d'** (Depot), **'c'** (Customer), or **'f'** (Recharging Station).

4.2.4 Multi-Depot Vehicle Routing Problem

The MDVRP variant was selected to evaluate the model’s ability to simultaneously assign customers to depots and optimize routes. This adds a „global“ layer of complexity, as the

heuristic must balance the workload across multiple starting locations while minimizing total fleet distance.

The experiments utilize two benchmark datasets established by **Cordeau [9]**, covering both the standard and time-constrained versions of the problem (MDVRP and Multi-Depot Vehicle Routing Problem with Time Windows (MDVRPTW)).

- **Dataset Overview:** The study utilizes the MDVRP set (instances p01–p23) and the **MDVRPTW** set (instances pr01–pr20). These instances feature between 2 and 9 depots and 48 and 360 customers.
- **Selection Motivation:** The Cordeau instances are selected because they represent the definitive test for „assignment-based“ routing. Unlike single-depot problems, success here requires the GP model to evolve features that account for a depot’s relative workload and the „distance advantage“ of one depot over another. Using both MDVRP and MDVRPTW allows a comparison of how time windows alter the optimal assignment logic in a multi-hub environment.
- **Dataset Format:** These datasets use a specific **Cordeau format** characterized by a complex header. The first line defines the problem type, the number of vehicles (m), the number of customers (n), and the number of depots (d). The customer data follows a table-based structure similar to Solomon, while a separate block at the end of the file defines the coordinates and operational limits (capacity and duration) for each individual depot.

4.3 Variant combinations

The modular architecture of this project is designed to handle all 16 possible combinations of the four primary VRP features: Capacity, Time Windows, Multi-Depot, and Green/Electric constraints ($2^4 = 16$). However, standardized datasets do not exist for every permutation, for instance, there is no widely recognized benchmark for a Multi-Depot Green VRP without Capacity constraints. To overcome this, the project utilizes data augmentation on the five selected base datasets. Here is a description of how each variant dataset is obtained:

- **CVRP:** Utilizes the **Augerat et al.** (Set A) benchmark.
- **VRPTW:** Utilizes the **Gehring and Homberger** large-scale benchmark.
- **G-VRPTW:** Utilizes the **Schneider et al.** electric vehicle benchmark.
- **MDVRP:** Utilizes the **Cordeau** multi-depot benchmark.
- **MDVRPTW:** Utilizes the **Cordeau** multi-depot benchmark with time windows.
- **GVRP:** Derived by removing the temporal constraints of the **Schneider** G-VRPTW dataset.
- **Green Vehicle Routing Problem with Time Windows and Multiple Depots (G-VRPTW-MD):** Synthetically generated by augmenting the **Schneider** dataset with additional depots. Depending on customer density, 1 to 4 extra depots are added (totaling 2 to 5). These are distributed circularly around the original depot with a radius defined as: $radius = 0.4 * \max(range_x, range_y)$

- **Green Vehicle Routing Problem with Multiple Depots (GVRP-MD):** Derived by removing the temporal constraints from the augmented G-VRPTW-MD dataset.

Note: For each of the eight variants listed above, uncapacitated versions are produced by removing the vehicle capacity constraints from the original or augmented files, completing the 16-variant problem space.

Table 4.2: Problem Variant Constraint Mapping

Variant Name	Green/Battery	Multi-Depot	Time Windows
VRP	0	0	0
VRPTW	0	0	1
MDVRP	0	1	0
MDVRPTW	0	1	1
GVRP	1	0	0
G-VRPTW	1	0	1
GVRP-MD	1	1	0
G-VRPTW-MD	1	1	1

4.4 Genetic Programming Terminals and Functions

4.4.1 Terminal Set

The terminal set consists of 19 features that represent the dynamic state of the routing process. These features serve as the building blocks for the evolved priority functions, allowing the model to evaluate the suitability of a candidate customer based on spatial, temporal, and resource-related constraints. To avoid useless terminals and to prevent the evolution of bloated trees, these features are modularly activated: each terminal category is included in the GP’s search space only if the corresponding problem variant (e.g., Time Windows, Green, or Multi-Depot) is enabled in the current experiment.

- **Spatial and Core Features:**

- *dist_to_depot*: Distance between the customer and the active route’s starting depot.
- *dist_from_current*: Euclidean distance from the vehicle’s current location to the candidate customer.
- *savings*: Calculated as $d(i, \text{depot}) + d(\text{depot}, j) - d(i, j)$, representing the Clarke-Wright savings for inserting the candidate into the current sequence.

- **Capacity and Load Management:**

- *demand*: The cargo requirement of the candidate customer.
- *remaining_capacity*: The current available volume in the vehicle prior to insertion.
- *load_percentage*: The ratio of current load to maximum vehicle capacity.

- **Temporal Dynamics:**

- *current_time*: The absolute time elapsed since the vehicle departed from the depot.
- *arrival_time*: The estimated time the vehicle would arrive at the candidate node.
- *ready_time*: The lower bound of the customer’s time window.
- *due_time*: The upper bound of the customer’s time window.
- *wait_time*: The time the vehicle would have to wait if it arrived now.
- *slack_to_due*: The time remaining between arrival and the customer’s due date.
- *route_urgency*: Calculated as *slack_to_due* divided by travel distance, a lower value indicates a higher scheduling pressure.

- **Energy and Sustainability:**

- *current_battery*: The absolute remaining energy in the vehicle’s battery.
- *battery_percentage*: The current state-of-charge (SoC) relative to maximum capacity.
- *energy_to_customer*: The predicted energy consumption required to reach the candidate from the current node.
- *dist_to_nearest_charger*: Distance from the candidate customer to the closest available charging station.

- **Multi-Depot Coordination:**

- *depot_rank*: The ordinal rank of the current depot relative to all other depots for this specific customer (1 = closest).
- *depot_distance_advantage*: The marginal distance saved by using the current depot compared to the second-nearest alternative.

Apart from the dynamic terminals described above, GP is also provided with a set of constants: $\{0.0, 1.0, -1.0\}$. These enable the evolved priority functions to perform basic arithmetic transformations.

4.4.2 Function Set

The function set defines the internal nodes of the evolved expression trees, determining how the 18 terminal features and constants are combined to calculate a priority score. The functions are selected to allow for non-linear relationships and logical branching within the priority heuristics.

- **Arithmetic Operators:** The set includes standard binary operators for addition (**add**), subtraction (**sub**), and multiplication (**mul**). These allow the GP to scale features and perform basic operations between terminals (e.g., *due_time*–*current_time*).
- **Protected Division:** To prevent runtime errors caused by division by zero, a *protected division* operator is implemented. If the absolute value of the divisor is less than 1×10^{-6} , the function returns a constant value of 1.0. This ensures that every evolved tree is mathematically valid and can be evaluated across all problem instances.

- **Comparison Primitives:** The set includes `min` and `max` functions. These primitives enable the GP to evolve „clamping“ logic or conditional-style selection without the overhead of explicit if-then-else statements. For example, a `min` function can be used by the model to prioritize whichever constraint *min(capacity, battery)* is currently the most restrictive.

4.5 Evolutionary Process Implementation

The evolutionary part of this project is built using the Distributed Evolutionary Algorithms in Python (DEAP) framework. This section will focus on the specific evolutionary operators implemented.

4.5.1 Initialization

At the start of the process, an initial population of `PrimitiveTree` objects is created. To guarantee a structurally diverse starting point, the Ramped Half-and-Half (genHalfAndHalf) method, described in subsection 3.3.2, is utilized. A depth range of $[2, 6]$ is used to initialize the trees.

The minimum depth of 2 is there to prevent the creation of „degenerate“ individuals consisting of only a single terminal, thereby ensuring that every initial solution represents a functional mathematical relationship.

The maximum initial depth of 6 provides sufficient structural headroom to explore complex feature interactions while remaining well below the global limit of 10, allowing for further growth during evolution. By „ramping“ the depth across this range, the engine samples a broad library of sub-expressions, maintaining the genetic diversity necessary to avoid premature convergence and providing a robust foundation for subsequent crossover and mutation operations.

4.5.2 Genetic Operators

New generations are produced by applying crossover and mutation operators to the current pool of individuals. Each time an individual is modified, its fitness values are invalidated to ensure it is later re-evaluated by the Fitness module.

- **Crossover:** For every pair of individuals in the offspring pool, crossover occurs with a probability of `cxpb`. The `cxOnePoint` operator selects a random branch in each parent and swaps them, allowing the GP to combine successful sub-expressions from different individuals.
- **Mutation:** To introduce new genetic material and avoid the possibility of premature convergence, the Uniform Mutation (`mutUniform`) is used with a probability of `mutpb`. When an individual is selected, the Uniform Mutation randomly replaces one subtree with a new expression generated by the `expr_mut` generator, which draws from the terminal and function sets. This generator uses the Ramped Half-and-Half method with a depth range of $[0, 3]$.

4.5.3 Selection and Elitism

The primary selection mechanism is **Tournament Selection**, which randomly selects a few individuals and then chooses the fittest for the next generation.

To ensure that the absolute best solution found throughout the entire evolutionary run is never lost, a specific Elitism strategy is implemented. At the end of each generation, the „Best of All Time“ individual is tracked. If this global champion is not already present in the current offspring, it replaces one randomly selected member of the population. This safeguard ensures a strictly non-decreasing performance trend over time.

4.5.4 Structural Constraints

While the initial population is generated with a depth limit of 6, subsequent genetic operations can lead to significant increases in complexity. To control for this, a hard **Maximum Depth** of 10 is enforced. Any tree exceeding this limit after crossover or mutation is pruned by replacing the over-limit branch with a single terminal.

4.6 Route Construction

The route construction mechanism serves as the interface between the evolved priority function and the VRP’s physical constraints. The design of this module is critical, as it dictates how the GP-generated scores are translated into feasible vehicle routes.

4.6.1 Construction Methodology

In the early phases of the project, a “post-construction penalty” strategy was attempted. This strategy works by allowing the system to construct any route, regardless of constraint violations, with the Fitness module applying penalties for each offense (e.g., missed Time window) during evaluation. However, this method proved ineffective for GP. This is due to the large number of invalid routes, resulting in a „flat“ fitness landscape in which the overwhelming majority of individuals were assigned identical, high-penalty scores. This is problematic, as it prevents the GP from identifying the varying levels of poor performance, effectively stalling the evolutionary process because it cannot detect a gradient towards better performance.

To resolve this, the system was transitioned to a Constrained Construction approach, inspired by the work of **Gil-Gala et al.** [12], which used a similar logic to evolve electric vehicle routing policies. In this reworked system, the construction algorithm considers only candidates that meet all active constraints at the time of selection, including vehicle capacity, time windows, and battery state of charge. This ensures that every generated route is feasible. As such, the Fitness module no longer needs to penalize invalidity and can focus exclusively on optimizing the total Euclidean distance. This change provides the necessary selective pressure for the GP to evolve sophisticated, efficient heuristics.

Serial vs. Parallel Construction

Vehicle routes can be constructed using serial, parallel, or semi-parallel strategies. In a serial approach, a single route is completed from start to finish before the next vehicle begins its journey. On the other hand, parallel construction expands multiple routes simultaneously by considering the best move across all active vehicles. The semi-parallel strategy uses the parallel method for a given number of vehicles (e.g., derived from total customer demand) and serves the remaining customers using the serial method.

For this implementation, a serial construction strategy was chosen. This decision is supported by the findings of Gil-Gala et al. [12], who observed that serial construction

generally produces superior results for GP-based heuristics. The serial method provides a more stable and predictable approach to the priority function, as it only needs to consider the context of a single growing route at each stage.

4.6.2 Pseudocode

Algorithm 4 outlines the primary Route Generation scheme together with the helper algorithms 5, 6, and 7, where algorithm 5 deals with depot selection, 6 checks the customers' feasibility, and 7 makes sure we can safely return to our starting depot.

Algorithm 4 Route Construction

Require: Instance I , Feature Extractor F , GP Scoring Function f_{GP}

Ensure: Set of routes $\mathcal{R}$

```

1:  $U \leftarrow$  Customers in  $I$ ;  $\mathcal{R} \leftarrow \emptyset$ ;  $D \leftarrow$  Depots in  $I$ 
2: Precompute nearest charging stations  $S$  and distances for all nodes
3: while  $U \neq \emptyset$  do
4:   // Initialize a new route
5:   if  $|D| > 1$  then ▷ Multi-depot
6:      $(\delta, c_0) \leftarrow \text{SelectBestDepotPair}(U, D, f_{GP})$  ▷ See Alg. 5
7:      $r \leftarrow [\delta, c_0]$ ;  $U \leftarrow U \setminus \{c_0\}$ 
8:     Update state  $(v, q, b, t)$  based on move  $\delta \rightarrow c_0$ 
9:   else
10:     $\delta \leftarrow D[0]$ ;  $r \leftarrow [\delta]$ 
11:    Initialize state:  $v \leftarrow \delta, q \leftarrow 0, b \leftarrow B, t \leftarrow 0$ 
12:   end if
13:   // Sequential Construction
14:   while True do
15:      $\mathcal{F} \leftarrow$  Find all feasible candidates in  $U$  ▷ See Alg. 6
16:     if  $\mathcal{F} = \emptyset$  then
17:       Close route  $r$  by returning to  $\delta$  (with charging if necessary)
18:       break
19:     end if
20:     Score each  $c \in \mathcal{F}$  using  $f_{GP}$  and extracted features from  $F$ 
21:      $c^* \leftarrow \arg \min_{c \in \mathcal{F}} (\text{score}_c)$ 
22:     Execute Move:
23:     if move to  $c^*$  is via station  $s$  then
24:       Append  $s$  to  $r$ ; Update  $t$  (travel + charging);  $b \leftarrow B$  ▷ Append station
25:     end if
26:     Append  $c^*$  to  $r$ ;  $U \leftarrow U \setminus \{c^*\}$ ; Update state  $(v, q, b, t)$  ▷ Append customer
27:   end while
28:    $\mathcal{R} \leftarrow \mathcal{R} \cup \{r\}$ 
29: end while
30: return  $\mathcal{R}$ 

```

The process begins with an initialization phase in which we extract information about the specific instance. It determines which variants to incorporate and how to distribute the nodes among customers, depots, and charging stations. During the precomputation stage,

the system calculates distances between all customers and stations, and identifies the K chargers closest to each customer and depot to optimize the following feasibility checks.

The main construction loop continues until all customers are served or no further customers can be served under our route-creation restrictions. If that happens, a penalty equal to four times the customer-to-depot distance is added to the individual’s fitness to discourage such solutions during the evolutionary process.

Route initialization varies significantly depending on the depot configuration. If we are using a single depot, what follows is a basic configuration of variables for the route, including the route’s starting point, time, load, and battery charge. This process gets much more complicated with multiple depots, because we must determine which depot will serve as our starting point. This process is described in Algorithm 5.

With the initial setup finished, a new loop for the sequential route construction begins. First, all customers that can be feasibly visited are identified; this is detailedly described in Algorithm 6. If no viable customers are found, the route is closed by returning the vehicle to its starting depot. Along this return trip, a charging station may be visited if the remaining battery charge is insufficient to reach the depot directly.

If there are multiple feasible candidates, the system scores each customer using the evolved GP priority function and the extracted features. This scoring method serves as the decision-maker, evaluating the immediate and long-term utility of the move based on the vehicle’s state and remaining demands. After that, the customer with the best score (in our case, the minimum) is chosen and added to the route. When a move is deemed feasible only via a charging station, that station is automatically added to the route before the customer’s visit.

Once the best candidate is selected, the vehicle state, including its current position, remaining capacity, battery level, and arrival time, is updated to reflect the move, and the process repeats until no further feasible expansions are possible. At this point, the current route is finalized and added to the solution set R . What follows is the initialization of a new route and vehicle, which repeats the route construction process starting from the depot selection logic.

Ultimately, this constructive process transforms the high-dimensional G-VRPTW-MD problem into a series of greedy, yet „safe,“ decisions guided by the GP. By the time the final customer is assigned, the engine has produced a complete set of routes that are not only feasible in terms of capacity and time windows but also in accordance with the integrated charging-station logic. This robustly constructed solution set is then returned to the main GP loop for fitness evaluation based on the total Euclidean distance.

Starting depot selection

For initial testing, this algorithm used a simple serial rotation between depots, but this method was quickly abandoned as it did not allow the GP to influence the choice of the starting depot. The current approach, detailed in Algorithm 5, evaluates all available and feasible customer-depot pairs using the evolved scoring function and selects the best one. The route is then initiated with the pair as its first two stops. This approach allows the GP to influence the starting depot choice using terminals like `depot_rank` and `depot_distance_advantage`, described in subsection 4.4.1.

Algorithm 5 Depot and First Customer Selection

```
1: function SELECTBESTDEPOTPAIR( $U, D, f_{GP}$ )
2:    $best\_score \leftarrow \infty$ ;  $best\_pair \leftarrow \text{null}$ 
3:   for each  $\delta \in D$  do
4:     for each  $c \in U$  do
5:       if move  $\delta \rightarrow c \rightarrow \delta$  is feasible then
6:          $\phi \leftarrow \text{ExtractFeatures}(c, \delta, \text{state}_0)$ 
7:          $score \leftarrow f_{GP}(\phi) + \text{Penalty}(\text{MaxTravelDist})$ 
8:         if  $score < best\_score$  then
9:            $best\_score \leftarrow score$ ;  $best\_pair \leftarrow (\delta, c)$ 
10:        end if
11:      end if
12:    end for
13:  end for
14:  return  $best\_pair$ 
15: end function
```

Feasibility Assessment

The essence of the building process lies in filtering potential customers, as shown in Algorithm 6. Instead of simply evaluating the immediate cost of a move, a multi-level check is conducted to assess its feasibility in terms of vehicle capacity, energy consumption, and time windows. If the corresponding variant isn't selected, each of these checks is either completely disabled or passes automatically with its default values.

For this function to work, it needs to know the current position, time, load, remaining battery, the depot the route originated from, and finally, the customer we are evaluating. The first check is a simple one to see whether the customer's demand, together with the current load, exceeds the maximum vehicle capacity. Now the code splits into two parts depending on how the client is reached.

The basic solution is for the vehicle to travel directly to the desired position. This is the preferred move provided the current battery level is sufficient. After checking the time window and ensuring we can safely return, as detailed in Algorithm 7, the function returns True with the direct suffix.

If the battery level is insufficient, and a direct route isn't feasible, a charging station needs to be inserted into the route. It would be computationally intensive and make little sense to try inserting every station for each customer, so only the K-nearest stations to the departing and arriving positions are evaluated. For these candidate stations, the algorithm validates the detour. It checks the battery required to travel to the station and then to the customer. It also checks the time windows with updated times from detours and charging, and performs a safe-return check, as in direct travel. The valid stations are then noted, and the one with the shortest travel path is returned to the main function.

If neither of these variants succeeds, the customer is labeled as unfeasible and excluded from the current route's candidate list, remaining in the unvisited set until a new assessment in the next step.

Algorithm 6 Feasibility Check

```
1: function ISFEASIBLE(position  $v$ , customer  $c$ , depot  $\delta$ , battery  $b$ , time  $t$ , load  $q$ )
2:   return false if  $q + \text{demand}(c) \leq \text{Capacity}$   $\triangleright$  Capacity constraint check
3:   // Option 1: Direct Move
4:    $d_{to\_c} \leftarrow \text{dist}(v, c)$ ;  $e_{dir} \leftarrow d_{to\_c} \cdot \rho$ 
5:   if  $b \geq e_{dir}$  and  $t + \text{time}(v, c) \leq \text{due}_c$  then
6:      $t_{dept} \leftarrow \max(t + \text{time}(v, c), \text{ready}_c) + \text{service}_c$ 
7:     if CHECKSAFEReturn( $c, b - e_{dir}, t_{dept}, d_r, d_{to\_c}, \delta$ ) then  $\triangleright$  See Alg. 7
8:       return true (Direct)  $\triangleright$  Feasible via direct path
9:     end if
10:  end if
11:  // Option 2: Move through a Charging Station
12:   $valid\_s \leftarrow \emptyset$ 
13:  for each  $s \in K\text{-nearest stations}(v) \cup K\text{-nearest stations}(c)$  do
14:    if  $b \geq \text{dist}(v, s) \cdot \rho$  and  $B \geq \text{dist}(s, c) \cdot \rho$  then
15:      Update  $t_{arr}$  and  $t_{return}$  via  $s$   $\triangleright$  Calculate timings through station  $s$ 
16:      return false if  $t_{arr} > \text{due}(c)$ 
17:      if CHECKSAFEReturn( $c, B - \text{dist}(s, c) \cdot \rho, t_{dept}, d_r, \text{dist}(v, s) + \text{dist}(s, c), \delta$ )
18:         $valid\_s \leftarrow valid\_s \cup \{s\}$   $\triangleright$  Station  $s$  is a viable detour
19:      end if
20:    end if
21:  end for
22:  return ( $|valid\_s| \geq 1$ ) ? true ( $best(valid\_s)$ ) : false
23: end function
```

Safe Return Assesment

Every potential move, whether direct or via a station, must pass a Safe Return verification outlined in Algorithm 7. This conservative look-ahead logic ensures that the vehicle never visits a customer if doing so would leave it unable to return to the starting depot without breaking any constraints.

The function identifies the nearest charging station to the customer and verifies that, even after serving the customer, the vehicle maintains sufficient energy to reach it. It also checks whether the vehicle can return to the depot from the station with a full battery and arrive in time before the depot's due date.

Furthermore, this check incorporates the maximum travel distance constraint. By calculating the route's cumulative distance, including the hypothetical return trip through a station, the system effectively removes any moves that would exceed the vehicle's operational range, as outlined in some of the instances.

This conservative check is not ideal, as it theoretically removes some potential moves that would be possible with a chaining of recharge stations. However, this would make the route construction algorithm overly complicated, and all available instances can be completed without requiring such complex multi-stop detours. This design choice maintains a balance between rigorous feasibility and the high-speed execution necessary for the iterative nature of GP.

Algorithm 7 Check Safe Return to Depot

```
1: function CHECKSAFEReturn(customer  $c$ , battery  $b_c$ , departure time  $t_{dept}$ , route distance  $d_r$ , distance to  $c$  :  $d_{to\_c}$ , depot  $\delta$ , stations  $\mathcal{S}$ )
2:    $s^* \leftarrow$  nearest charging station to  $c$ 
3:    $d_{cs} \leftarrow \text{dist}(c, s^*)$ ,  $d_{s\delta} \leftarrow \text{dist}(s^*, \delta)$ 
4:    $e_{cs} \leftarrow d_{cs} \cdot \rho$ ,  $e_{s\delta} \leftarrow d_{s\delta} \cdot \rho$ 
5:   if  $b_c < e_{cs}$  then ▷ Cannot reach station from customer
6:     return false
7:   end if
8:   if  $B < e_{s\delta}$  then ▷ Full battery cannot reach depot from station
9:     return false
10:  end if
11:   $d_{total} \leftarrow d_r + d_{to\_c} + d_{cs} + d_{s\delta}$ 
12:  if  $d_{total} > D_{max}$  then ▷ Exceeds maximum travel distance
13:    return false
14:  end if
15:   $t_\delta \leftarrow t_{dept} + d_{cs} + \text{ChargeTime}(b_c - e_{cs}) + d_{s\delta}$ 
16:  if  $t_\delta > \text{due}_\delta$  then ▷ Violates depot time window
17:    return false
18:  end if
19:  return true
20: end function
```

4.6.3 Feature Extraction and Scoring

The feature extraction process is hybridized, meaning it is carried out using both the **FeatureExtractor** class and the primary construction loop. This ensures that the scoring function has access to both the raw node information and information that is only computed at customer selection time.

Hybrid Feature Assembly

The features used in the scoring function f_{GP} are assembled in two stages:

1. **The Extraction Layer:** At the start the `feature_extractor.extract_features` method creates a primary feature set. This set includes node-specific data, such as `slack_to_due` and `route_urgency`, depot-specific metrics like `depot_rank`, and for GVRP variants `battery_percentage` and `dist_to_nearest_charger`.
2. **The Construction Layer:** After the main extraction, the route construction algorithm provides the dynamic context necessary for some of the features to work properly. This includes the actual travel distance of the current move, the `savings` value, the `depot_relative_workload`, and the `remaining_capacity`.

The Scoring Mechanism

Once feature assembly is complete, the values are passed to the evolved scoring function, f_{GP} . In the case of invalid results, such as when a division by zero or a floating-point overflow happens, a huge penalty score (10^6) is applied to the individual. This prevents edge

cases from crashing the heuristic and allows the evolutionary process to discard unstable scoring functions.

4.7 Baseline Heuristics Implementation

4.7.1 Nearest Neighbour

The NN implementation uses the route-construction algorithm, which simply replaces the scoring function with the current distance to the candidate customer (`dits_from_current`), so the solver always prefers the nearest feasible customer.

4.7.2 Clarke-Wright Savings

This project’s implementation of the Clarke-Wright Savings Heuristic follows the same principle as the original, but unlike the NN heuristic, it is heavily edited, as the original had no way to handle our many variants. The most problematic part of the implementation are the charging stations and the choice of the starting depot for all customers.

The procedure begins by assigning each customer c to its nearest feasible depot $d \in D$. At the start, a preliminary set of routes is generated, each comprising a single trip: $d \rightarrow c \rightarrow d$. The core of the algorithm is the calculation of a Battery-Aware Savings Matrix. For every pair of customers (i, j) assigned to the same depot, the Actual Saving is calculated as:

$$S_{ij} = \text{Cost}(d, i, d) + \text{Cost}(d, j, d) - \text{Cost}(d, i, j, d)$$

The Cost function is not a simple distance calculation. It is the result of a full route validation that inserts charging stations where needed. A merge is only considered if the combined route satisfies vehicle capacity, hard time windows, and battery reachability (including a safety buffer to the nearest charger). The algorithm then iteratively merges routes with the highest positive savings, provided the customers are at the exterior ends of their respective routes, and the merge remains feasible under all constraints.

With this method, the Savings heuristic works for all of our variants and, on average, heavily outperforms the NN heuristic across all tested instances, except for the multiple-depot VRP without capacity constraints, as shown in Table 4.3. This is likely the product of an unoptimal assignment of customers to a specific depot. However, across all other variants, the advantage of the Savings heuristic is clear, as it provides a superior foundation for further optimization by prioritizing long-term route feasibility over immediate proximity.

Table 4.3: Heuristic Performance Comparison: Nearest Neighbor vs. Savings

Variant	Capacity: N		Capacity: Y	
	NN Avg	Savings Avg	NN Avg	Savings Avg
VRP	687.59	603.00	1453.26	1094.05
MDVRP	2250.19	2318.15	3991.60	2597.10
GVRP	658.17	606.45	783.89	657.35
GVRP-MD	655.52	572.09	728.94	588.77
VRPTW	9400.97	5986.77	9509.66	6095.62
MDVRPTW	4015.06	3299.57	4116.90	3302.71
G-VRPTW	1308.40	1106.76	1308.06	1107.13
G-VRPTW-MD	1119.04	757.92	1119.02	758.02

Algorithm 8 Savings Heuristic

Require: Instance I , Capacity Q , Battery B

Ensure: Set of feasible routes $\mathcal{R}$

```
1: Precompute:  $K$ -nearest stations for all nodes
2: Initialize: Assign each customer  $c$  to the nearest feasible depot  $\delta_c$ 
3: Create initial routes  $r_c = [\delta_c, c, \delta_c]$  for all  $c \in \text{Customers}$ 
4:  $S \leftarrow \emptyset$  ▷ Savings list
5: for each pair  $(c_i, c_j)$  sharing depot  $\delta$  do
6:    $Cost_{indiv} \leftarrow \text{Validate}([\delta, c_i, \delta]) + \text{Validate}([\delta, c_j, \delta])$ 
7:    $Cost_{merge} \leftarrow \min(\text{Validate}([\delta, c_i, c_j, \delta]), \text{Validate}([\delta, c_j, c_i, \delta]))$  ▷ Try both orders
8:   if  $Cost_{merge} < \infty$  then
9:      $S \leftarrow S \cup \{(c_i, c_j, Cost_{indiv} - Cost_{merge})\}$ 
10:  end if
11: end for
12: Sort  $S$  by savings value in descending order
13: for each  $(c_i, c_j, value) \in S$  do
14:    $r_1, r_2 \leftarrow$  routes containing  $c_i$  and  $c_j$ 
15:   if  $r_1 \neq r_2$  and  $c_i, c_j$  are at route endpoints then ▷ Standard CW constraint
16:      $r_{new} \leftarrow \text{Merge}(r_1, r_2)$  ▷ Join at endpoints
17:     if  $\text{VALIDATE}(r_{new})$  then
18:       Update  $\mathcal{R}$  by replacing  $r_1, r_2$  with  $r_{new}$ 
19:     end if
20:   end if
21: end for
22: return  $\mathcal{R}$ 
```

Chapter 5

Experimentations and Results

5.1 Experimental Setup

This section describes the configuration and methodology used throughout the experimental phase, including the baseline hyperparameters, evaluation protocol, computational budget, and the hardware environment in which all experiments were conducted.

5.1.1 Initial Configuration

Before beginning any experiments, the default configuration of the evolutionary parameters must be specified. The evolutionary parameters represent the standard evolutionary environment in which the initial experiments were conducted prior to the specification of the optimal parameters during the experimentation process.

The evolutionary process was carried out with a population size of 200 and with 50 generations. To ensure the heuristics were general enough across different scenarios, 5 training instances were utilized per variant. Selection pressure was regulated using a tournament size of 3. To maximize genetic diversity in the population and encourage exploration of the search space, the crossover rate was set to 1.0, and the mutation rate to 0.2.

The population was initialized with a depth range of 2 – 6 to introduce structural diversity in the first generation. For mutation, the subtree depth was restricted to 0 – 3. A global maximum depth of 10 was maintained.

Table 5.1 summarizes the complete set of hyperparameters.

Table 5.1: Baseline GP Hyperparameters

Parameter	Value
Population Size (P)	200
Number of Generations (G)	50
Training Instances ($ T_{train} $)	5
Tournament Size	3
Crossover Rate	1.0
Mutation Rate	0.2
Initial Population Depth	2–6
Mutation Subtree Depth	0–3
Maximum Tree Depth	10

5.1.2 Evaluation Protocol

To ensure generalization of the developed heuristics, all experiments were conducted across the full range of 16 routing variants. This is because incorporating all variants into our experiment aims to prevent the search procedure from becoming overly specialized to particular problem instances.

To account for the stochastic nature of our GP algorithm, each experiment configuration was run 5 times with random starting seeds.

5.1.3 Search Budget

To maintain a fair comparison between GP configurations, a fixed computational budget was established. For any given problem variant within an experiment, the GP was restricted to a maximum of 50,000 fitness evaluations. It is defined as:

$$B_{variant} = |T_{train}| \times P \times G \leq 50,000 \quad (5.1)$$

where $|T_{train}|$ is the number of training instances, P is the population size, and G is the number of generations. Since each experiment involves searching through all 16 problem variants, the total search budget per experiment is 800,000 fitness calls.

5.1.4 Planned Experiments

The experimental phase was designed to isolate the effects of GP parameters. These are the planned experiments:

1. **Search Dynamics:** Studying the best ratio between population size (P) and the number of generations (G) while keeping $B_{variant}$ constant.
2. **Genetic Operators:** Testing the sensitivity of the evolutionary process to various mutation and crossover rates.
3. **Selection Pressure:** Analyzing different Tournament Sizes in order to find the optimal trade-off between exploration and convergence rate.
4. **Structural Constraints:** Investigating the influence of both minimum and maximum Tree Depth for both initial and mutated trees.
5. **Training Data Scaling:** Determining the effect of the training set size ($|T_{train}|$) on the ability of the GP to evolve generalizable logic.
6. **Terminal Set:** Learning the effects of removing unused terminals from the available set.

5.1.5 Technology

The workload was distributed across four dedicated nodes at the Faculty of Information Technology, Brno University of Technology (FIT VUT). Specifically, the experiments were executed on the `edesign1` through `edesign4.fit.vutbr.cz` servers.

While hardware configurations vary across the cluster, the primary node (`edesign1`) is equipped with two Intel Xeon E5-2630 v4 processors (20 physical cores and 40 threads per node) and 128 GB of RAM. The systems run Devuan GNU/Linux 5 (Daedalus).

5.2 Hyperparameter Analysis

5.2.1 Search Dynamics: Population vs. Generation Ratio

In order to find the optimal ratio between population size (P) and the number of generations (G), four ratios were tested using the same fixed budget as in Section 5.1.3. This experiment investigates how the search would profit from either a wide search initially (large P) or a detailed search from a smaller genetic pool (large G).

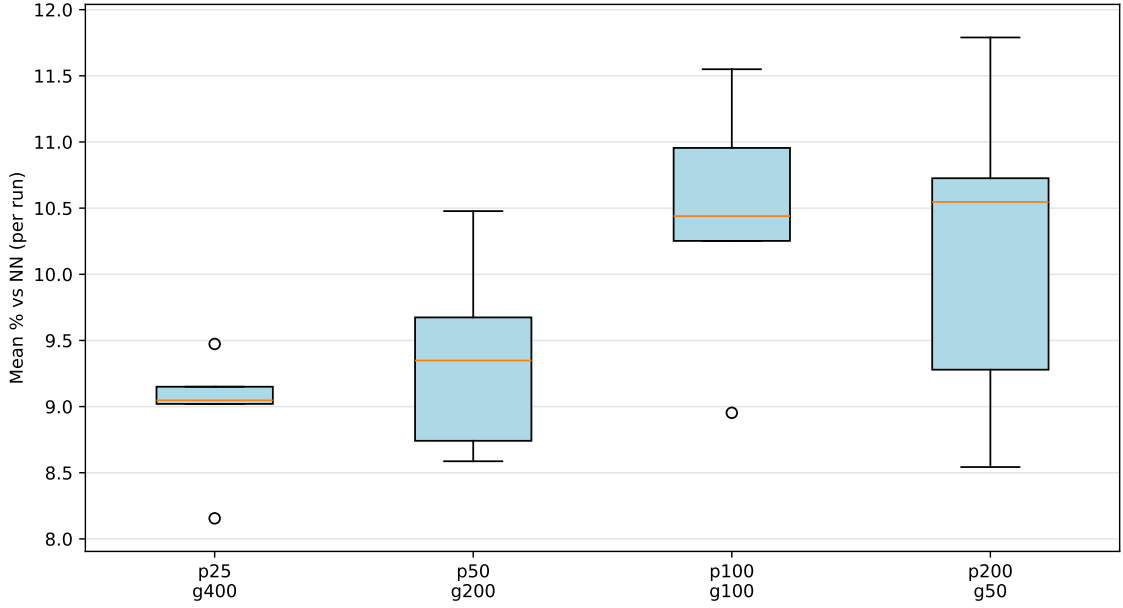


Figure 5.1: Impact of population and generation sizes on mean % improvement over NN.

Figure 5.1 presents the results across all 16 variants using unweighted means. Every tested configuration successfully outperformed the NN baseline. However, the performance differences between the four $P : G$ ratios were relatively small, with a 1.46% margin separating the highest and lowest mean results. Within this range, the **p100 g100** configuration achieved the greatest overall improvement of +10.43% (± 0.97).

The boxplot highlights a clear trade-off between population diversity and search stability. The p200 g50 variant achieved a competitive median, but its interquartile range (IQR) and standard deviation (1.28) indicate that 50 generations are insufficient to consistently refine a large population. In contrast, p100 g100 offers a more concentrated distribution of results, suggesting a more reliable convergence. The configurations p25 and p50 were the most stable but converged to worse solutions ($\approx 9\%$), due to insufficient structural diversity.

Figure 5.2 shows the convergence behavior of each configuration across a shared budget of fitness evaluations, with shaded regions indicating 95% confidence intervals. As absolute fitness values differ considerably across variants, larger instances dominate the pooled mean, so the graph provides a summary rather than a precise comparison. With that in mind, the curves reinforce the boxplot findings: smaller-population configurations, such as p25 g400, start higher but quickly narrow the gap through a steeper initial decline. The p100 g100 setup separates itself as the leader around 8,000 evaluations, while the second-best p200 g50 requires roughly 20,000 evaluations to distance itself from the remaining configurations.

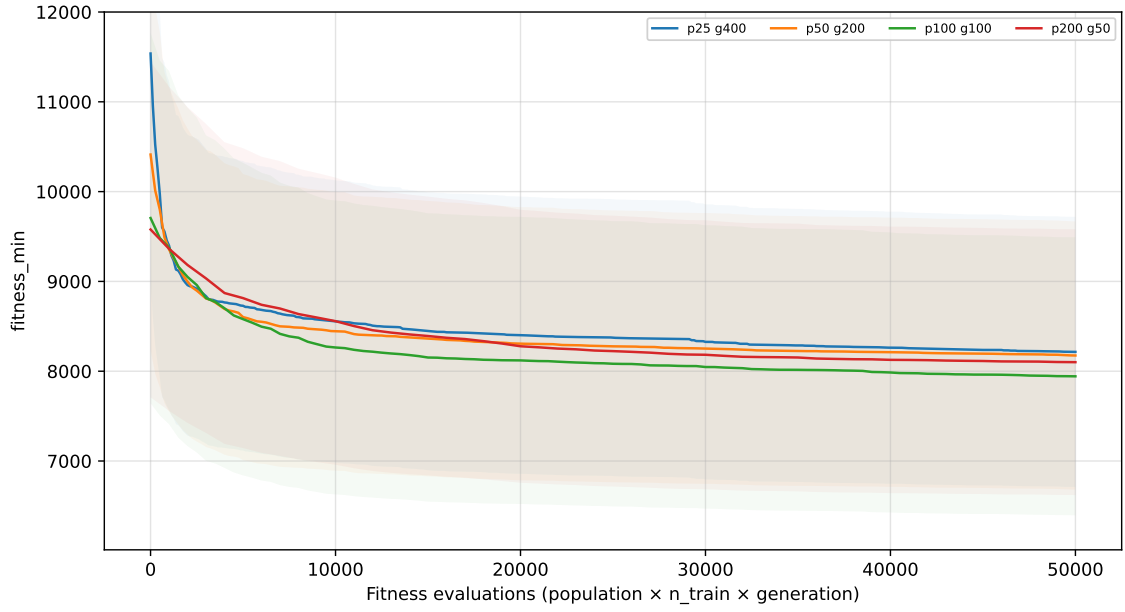


Figure 5.2: Convergence curves for each population/generation configuration pooled across all variants.

This suggests that the **p100 g100** setting provides the optimal trade-off between diversity and intensification, allowing for more efficient exploration of the complex search space than the more extreme population-generation ratios.

Figure 5.3 provides a breakdown of performance based on specific problem features. The GP-evolved heuristics achieve the best performance gains for the Time Windows variant. Meanwhile, the Multi-Depot variants proved the most challenging and yielded the lowest improvement compared to the NN baseline. The heatmap confirms that the **p100 g100** configuration provides the most consistent performance gains across all constraint types. This ratio successfully balances capacity and time-window constraints while preventing performance degradation across multiple depots.

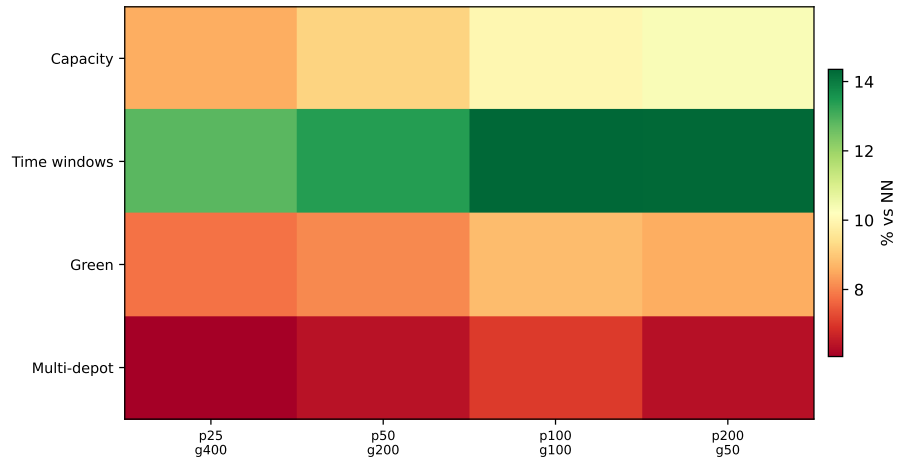


Figure 5.3: Mean improvement over NN categorized by problem constraints.

Given its superior mean and higher stability compared to the p200 variant, p100 g100 was selected as the fixed baseline configuration. Further experiments in this study were run with this population-to-generation ratio.

5.2.2 Genetic Operator Sensitivity: Crossover vs. Mutation

The efficiency of the GP search largely depends on the values assigned to the (*cx*) and mutation (*mu*) parameters. While crossover drives the search process by exploiting available genetic information, mutations maintain diversity by preventing premature convergence. We have tested five parameter combinations to find the optimal settings within the VRP search space.

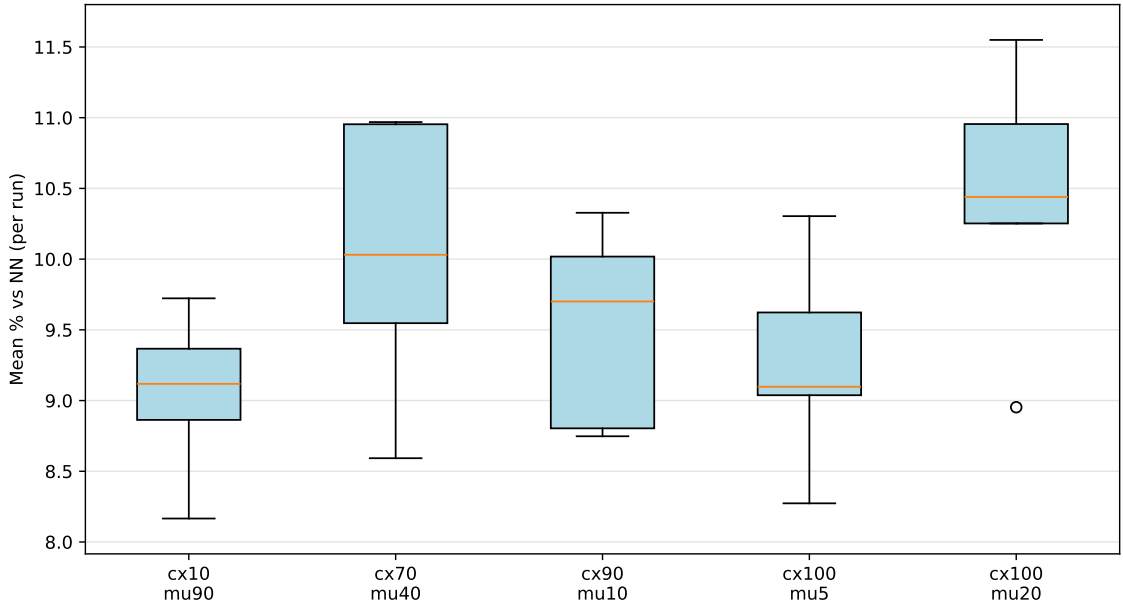


Figure 5.4: Impact of crossover and mutation rates on mean % improvement over NN.

Based on the results presented in Figure 5.4, the default combination of $cx=1.0$ and $mu=0.2$ (cx100 mu20) remains the best choice, achieving an unweighted mean improvement of $+10.43\%$ (± 0.97).

Several conclusions can be drawn regarding the operator dynamics:

- **Crossover Dominance:** The configuration relying almost exclusively on mutation (cx10 mu90) resulted in the lowest performance ($+9.05\%$), demonstrating that route optimization is more efficient when it recombines known solutions rather than creating new random structures.
- **Instability:** Although cx70 mu40 provides quite a promising mean ($+10.02\%$), it has the highest interquartile range among all the tested options, which implies increased variability caused by the excessive use of the mutation operator (0.4).
- **The Importance of Mutation:** The performance of the search algorithm was negatively impacted with the reduction of the mutation parameter value to a bare minimum (cx100 mu5), leading to a $+9.27\%$ decrease in the performance score as well as a highly skewed distribution favoring lower improvements.

The ranking of results demonstrates that while crossover is essential for building complex logic, a mutation rate of 0.2 provides the necessary „noise“ to maintain a productive search. As a result, the rates of 1.0 for crossover and 0.2 for mutation are maintained for the remainder of this study.

5.2.3 Selection Pressure: Tournament Size

Selection pressure, controlled by the tournament size (t), determines the probability with which the highest-fitness individuals are selected for reproduction. The larger the tournament size, the greater the selection pressure, driving the population towards more greedy exploration of the search space. With smaller tournaments, genetic diversity will be maintained at higher levels, but convergence will be slower. We experimentally studied four tournament sizes ($t \in \{2, 3, 5, 7\}$) and found the best parameter value for our routing algorithm.

The baseline parameter of $t=3$ remains the optimal choice, providing an increase in mean performance of **+10.43%** (± 0.97). As can be seen from Figure 5.5, results prove that our search space is sensitive to selection pressure:

- **Insufficient Pressure ($t=2$):** The decrease in tournament size down to 2 resulted in a performance degradation (+9.64%) and a higher variance, visible in the stretched boxplot. This suggests that with very low pressure, the search becomes too stochastic, failing to consistently propagate high-quality solutions through the generations.
- **Excessive Pressure ($t=5, t=7$):** Increasing the tournament size did not lead to better results. While $t=7$ achieved a competitive median, its performance was less stable than $t=3$. This suggests that stronger selection pressure forces the population to quickly find local optima, preventing it from reaching the global one.

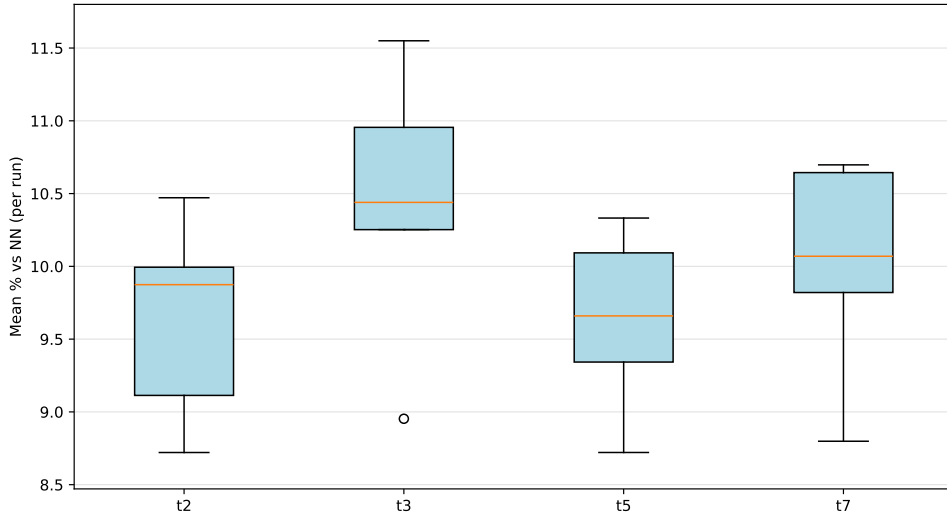


Figure 5.5: Impact of tournament size on mean % improvement over NN.

The obtained data show that a moderate selection pressure of 3 is sufficient to drive the search towards high-quality logic while maintaining the required diversity to handle 16 different problem variants. Therefore, $t=3$ will be used for heuristic evolution.

5.2.4 Expression Size

This experiment investigates the interaction between the initial population depth (starting complexity) and the mutation subtree depth (complexity added during search). By analyzing these constraints, we aim to identify the optimal balance between providing the GP with enough „building blocks“ for complex logic while mitigating structural bloat.

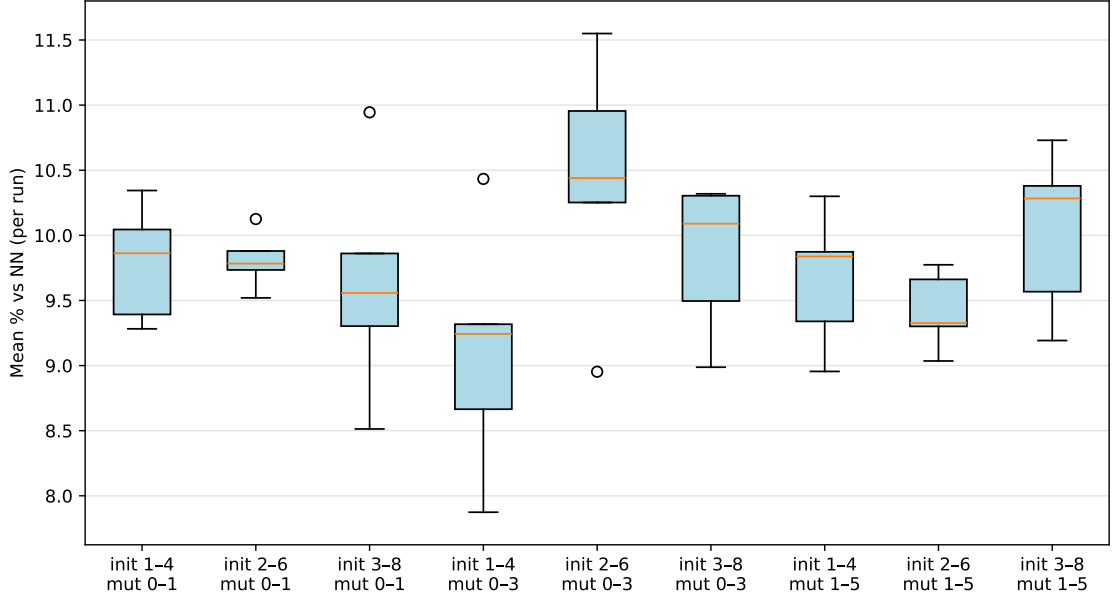


Figure 5.6: Impact of initialization and mutation depth on mean % improvement over NN.

The results in Figure 5.6 confirm that the baseline (`init 2-6/mut 0-3`) is the most effective configuration out of the ones tested. Furthermore, the data indicate a functional dependency on the combination of initialization and mutation depth, rather than on either alone.

The most significant performance drop occurs in the `init 1-4/mut 0-3` configuration. Its low median ($\approx 9.2\%$) and considerable negative spread suggest that starting with very shallow trees while allowing moderate mutation leads to unstable convergence to poor local optima.

Restricting mutation to `mut 0-1` creates a bottleneck for all tree sizes, as the search lacks the ability to refine initial logic effectively. In contrast, while deep initial trees (`init 3-8`) perform poorly with limited mutation, they achieve the best performance within the `mut 1-5` group, indicating that complex initial structures require significant mutation to be successfully optimized.

Evolution of Heuristic Size

The structural growth patterns for the developed heuristics are provided in Figures 5.7, 5.8, and 5.9. Each figure shows the number of nodes averaged over 100 generations, demonstrating how the structural complexity of the generated rules depends on the initialization depth and mutation restrictions.

All experiments follow a persistent trend of increasing node counts, indicating structural bloat. The growth rate is dependent on the mutation depth parameter. In the `mut 1-5`

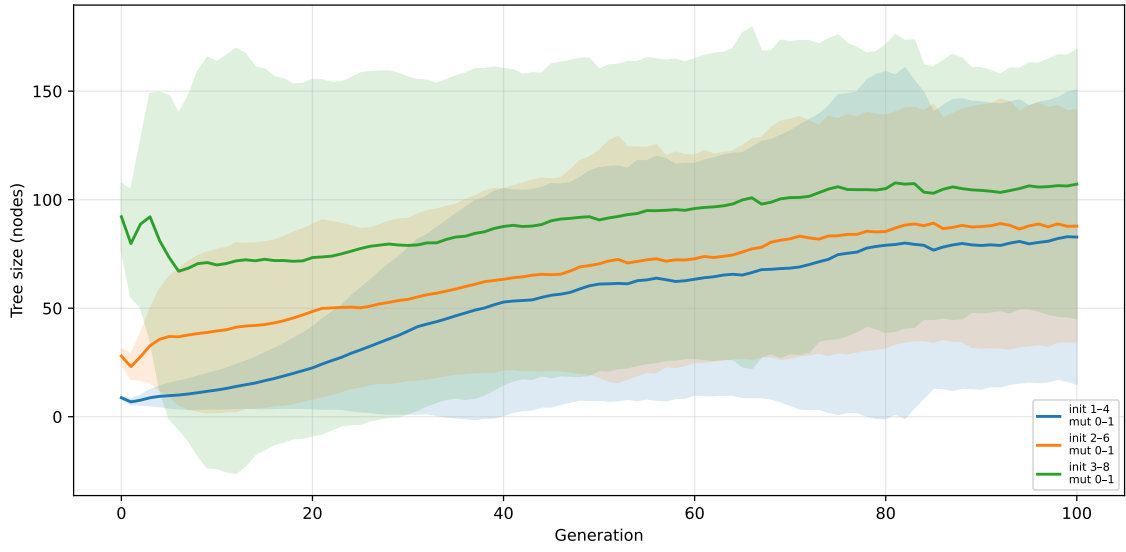


Figure 5.7: Tree size evolution (nodes) for Mutation Depth 0-1.

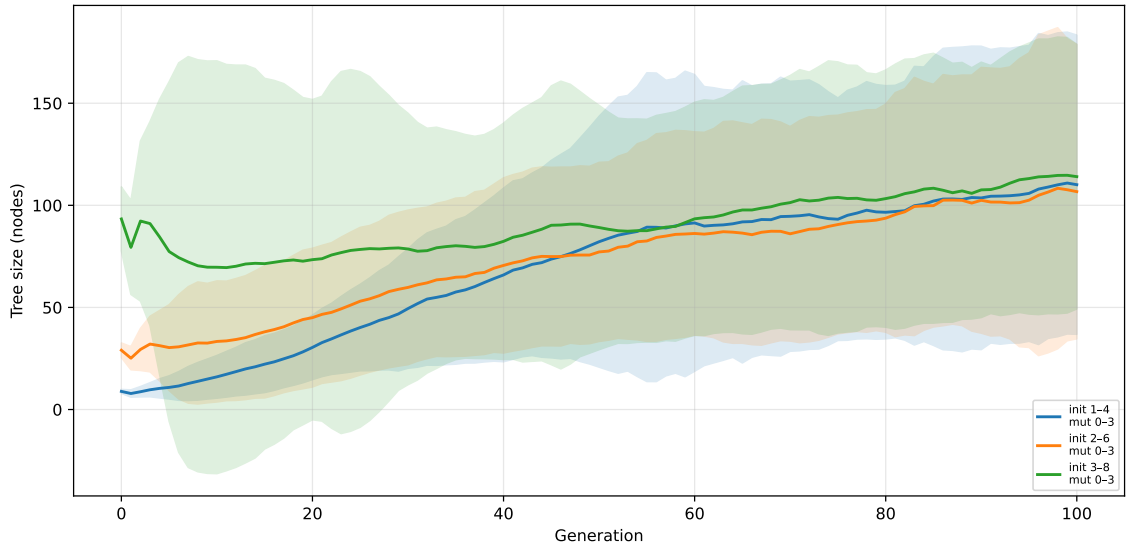


Figure 5.8: Tree size evolution (nodes) for Mutation Depth 0-3.

setting (Figure 5.9), the search behaves aggressively and produces much higher average tree sizes of 120–145 nodes, with a higher variance, compared to the others. In contrast, the `mut 0-1` setting (Figure 5.7) has the least growth, with all configurations finishing between 80 and 105 nodes.

There is an interesting structural consistency across the tracks in the `mut 0-3` setting (Figure 5.8), all configurations finish at about the same size of 110 nodes, regardless of their initial structures. However, despite their similar node counts, these tracks differ significantly in performance, as shown in Figure 5.6. This indicates that late-generation growth cannot retroactively bridge the gap in heuristic quality established by a more diverse initial population.

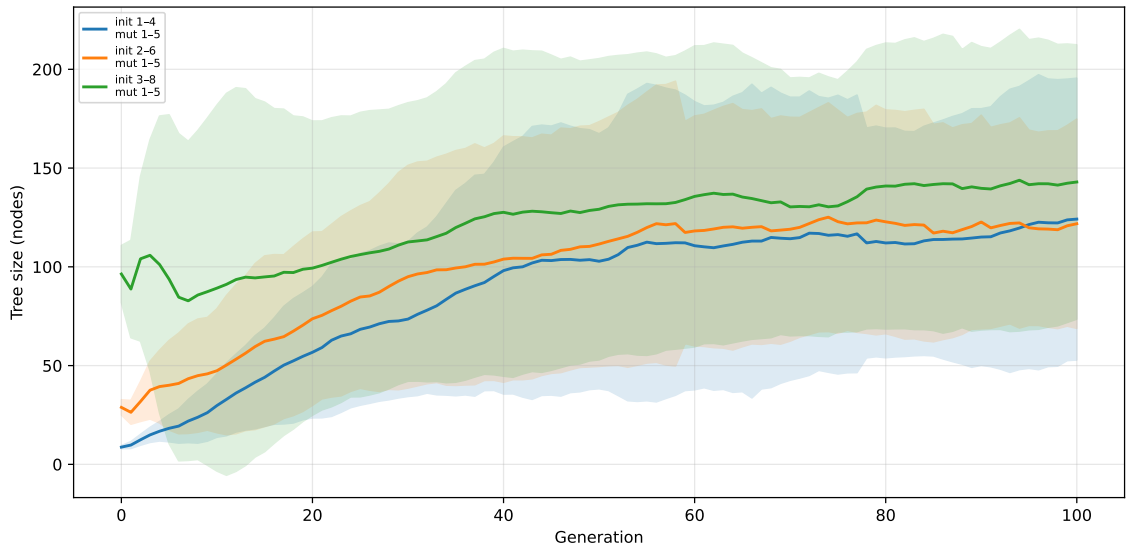


Figure 5.9: Tree size evolution (nodes) for Mutation Depth 1–5.

Correlation between Size and Performance

Figure 5.10 plots the relationship between the tree size of the best-performing individual for each problem variant per run and its corresponding improvement over the NN baseline.

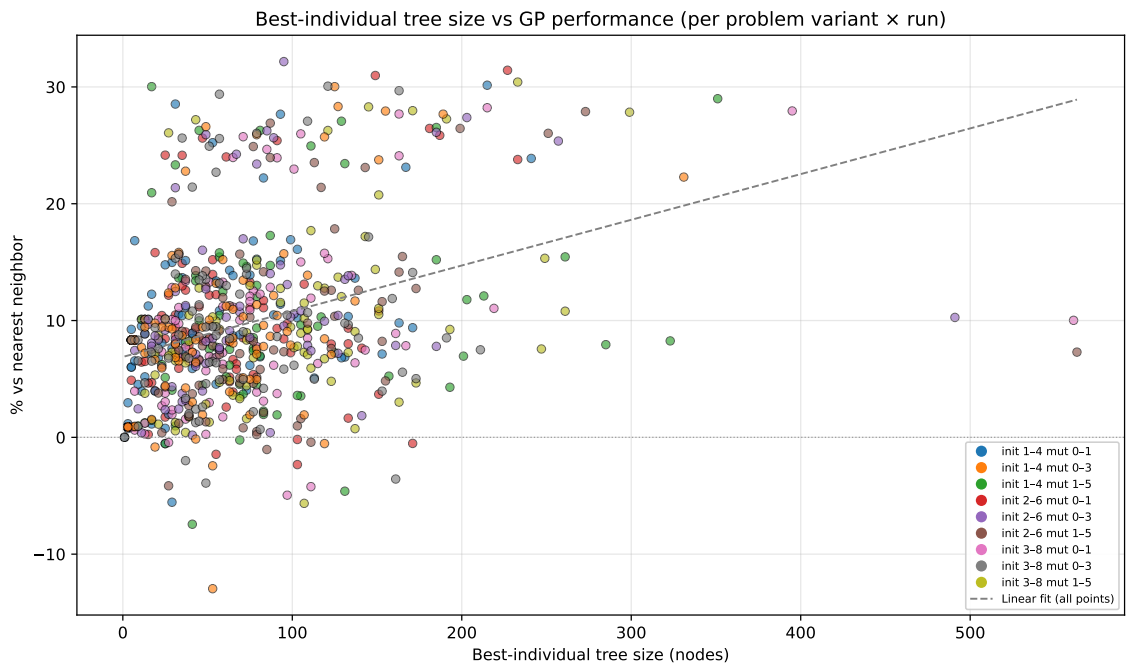


Figure 5.10: Correlation between the tree size of best-performing individuals (per variant $\times$ run) and percentage improvement over NN.

Even though the positive trend of the linear model shows that an increased number of nodes yields greater logical capability to handle routing complexity, the large variation in

data points indicates that size is a very unreliable metric for predicting good logic. In fact, high-quality heuristics ($> 20\%$ improvement) are frequently found at sizes < 100 , whereas extreme outliers, some exceeding 450 nodes, fail to outperform these smaller expressions.

The `init 2-6/mut 0-3` baseline will be retained for future experiments as it consistently locates high-quality logic within a practical size range, avoiding the computational overhead and noise associated with non-functional bloat.

5.2.5 Training Set

This section examines the trade-off between the number of training instances and the depth of the evolutionary search process. To provide a comparable environment, the same total search budget as before was used. Thus, with an increase in the number of training samples, the number of individuals (P) and generations (G) decreased ($P = G = \sqrt{50,000/n_{train}}$). Specifically, the investigated configurations included **train 1** ($P, G = 223$), **train 3** ($P, G = 129$), **train 5** ($P, G = 100$), and **train 10** ($P, G = 70$).

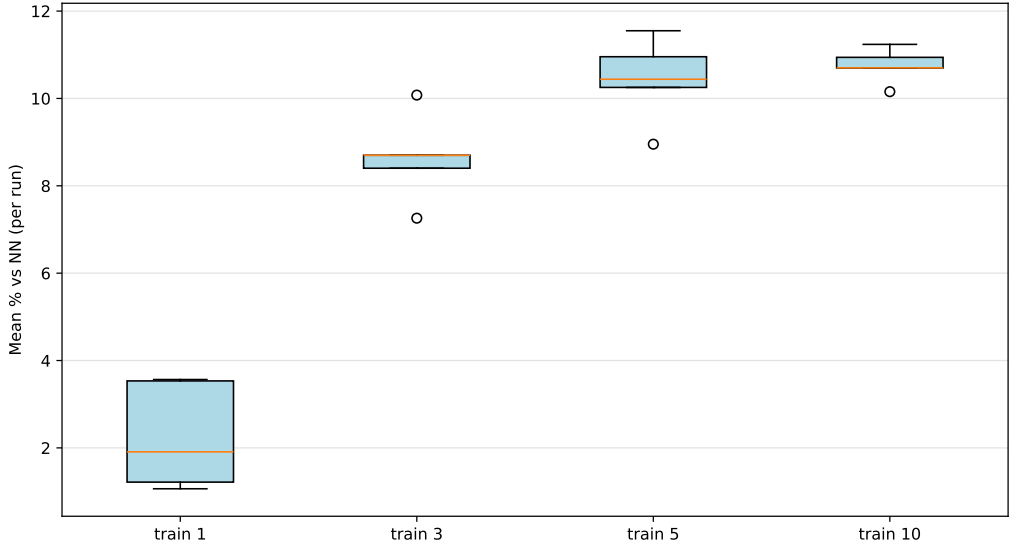


Figure 5.11: Impact of training set size on mean % improvement over NN.

The results in Figure 5.11 demonstrate a strong correlation between training set size and the generalization capability of the heuristics:

- **Overfitting in Minimal Sets (train 1):** With only one training sample used, the performance of the algorithm dropped significantly ($+2.257\% \pm 1.221$) and resulted in a high variance. With just one problem variant available, the GP is heavily biased towards the logic specific to that single instance, not creating anything that would apply in a general case.
- **The Stability of Larger Training Sets:** As more instances for training become available, a considerable boost in the system performance becomes noticeable. The **train 3** variant managed to produce the mean improvement $+8.627\% \pm 1.004$, **train 5** reached $+10.430\% \pm 0.967$, and the largest **train 10** managed to produce a total gain of $+10.746\% \pm 0.398$. We also observe a reduction in standard deviation, confirming that a broader range of instances prevents the search from chasing outlier logic.

- **Computational Cost:** While the **train 10** setup has demonstrated outstanding performance, it comes with a non-linear computational cost. Despite using exactly the same number of evaluations as **train 5**, the runs associated with **train 10** have used 25% more computational time. The reason is the use of larger problem instances in the extended training set, which increases the complexity of the routing simulation.

The experimental study clearly showed the need for diversity in training sets to solve the proposed problem. While the **train 10** configuration takes longer to run, its superior peak performance and stability make it the most reliable baseline for further optimization. Therefore, **train 10** is used for the final experiment, an analysis of terminal usage and their removal. This ensures that the experiment will be based on the most capable heuristics evolved in this study.

5.3 Terminal Set Analysis

Following the hyperparameter tuning, we will now discuss the nature of the search space. This experiment aims to explore the effect of removing underused terminals on the evolutionary search process. By isolating the most impactful features, we aim to refine the terminal set to accelerate convergence and improve heuristic quality. A detailed description of each terminal is provided in Section 4.4.1

5.3.1 Utilization Analysis

An initial analysis of terminal frequency was conducted using the baseline configuration. Figure 5.12 illustrates the total count of each terminal appearing in the best-performing individuals across all problem variants and runs.

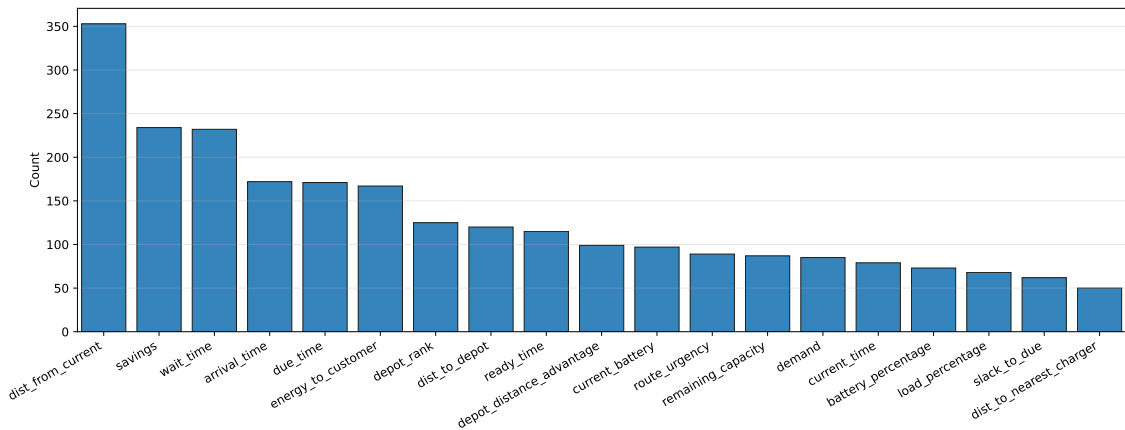


Figure 5.12: Frequency of terminal usage within the best-performing individuals

These frequencies are influenced by the availability of each feature: while **savings**, **dist_from_current**, and **dist_to_depot** are present in all 16 variants, the other terminals appear only in the 50% of cases where their associated problem constraints are enabled.

Despite their limited availability, terminals that correspond to time windows such as **wait_time**, **arrival_time**, and **due_time** have a very high utilization rate. The appearance of these terminals in the top five reflects their significance in developing an appropriate

routing logic that satisfies temporal constraints. On the other hand, terminals handling vehicle capacity have the lowest utilization, indicating they are the primary candidates for pruning, as their contribution is negligible even when the respective constraints are active.

5.3.2 Terminal Subsets

Based on the usage data, four distinct terminal sets were defined to test the hypothesis that a more focused search space leads to superior performance. These sets are detailed in Table 5.2.

Table 5.2: Terminal set configurations across different pruning levels.

Terminal	Full	Refined	Core	Atomic
dist_from_current	x	x	x	x
savings	x	x	x	x
wait_time	x	x	x	x
arrival_time	x	x	x	x
due_time	x	x	x	x
energy_to_customer	x	x	x	x
depot_rank	x	x	x	
dist_to_depot	x	x	x	
ready_time	x	x	x	
depot_dist_adv	x	x		
current_battery	x	x		
route_urgency	x	x		
remain_capacity	x	x		
demand	x	x		
current_time	x	x		
battery_pct	x			
load_pct	x			
slack_to_due	x			
dist_to_charger	x			

In order to examine the effects of complexity on search space performance, the terminal set is turned into four with decreasing complexity:

- **Refined Set:** The four least-active terminals (usage count below 75) were removed, trying to eliminate clear noise.
- **Core Set:** This configuration drops 7 more terminals and retains only those used more than 100 times.
- **Atomic Set:** This set utilizes only the top six elements found to constitute the foundations of the evolved VRP heuristics.

By analyzing these sets, we aim to identify the point at which the GP has sufficient information to address the constraints without being slowed by underutilized data points.

5.3.3 Results and Discussion

Based on the findings of the pruning experiments, it can be concluded that the hypothesis regarding better heuristic performance within a more limited search space is correct. With more specialized terminal sets, better results were observed, suggesting that underutilized terminals degraded search efficiency.

Overall Performance and Robustness

The best result was achieved by the Refined configuration with a mean improvement of $+11.171\% \pm 0.679$. As shown in Figure 5.13, this variant had a rather high variance compared to the others.

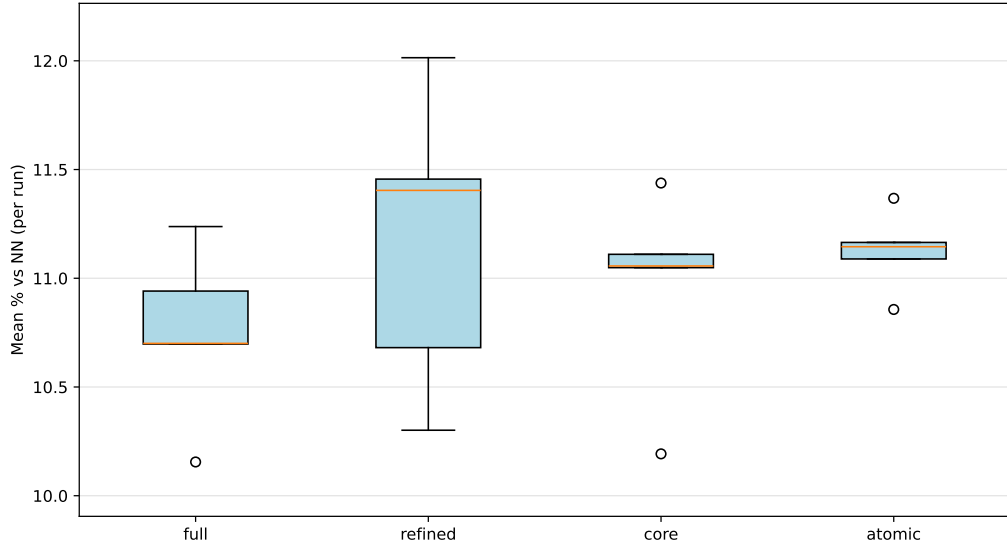


Figure 5.13: Impact of different terminal sets rates on mean % improvement over NN.

In contrast, the Atomic configuration (six terminals only) produced results that were almost identical ($+11.125\% \pm 0.183$), with much greater consistency. The extremely small standard deviation clearly shows that a minimalistic search space creates a more predictable landscape for the Genetic Programming algorithm, which allows it to consistently converge to the optimal solution. The next-best result was achieved by the Core setting, which produced an improvement of $+10.969\% \pm 0.463$. At last, the worst-performing configuration was the Full baseline, with an improvement of $+10.746\% \pm 0.398$.

Constraint-Specific Analysis

The impact of terminal elimination varied across the problem variants. Figure 5.14 visualizes the performance of each set across the four primary constraint categories.

For Capacity-constrained variants, the best performance was found in the Refined set. The deterioration in performance observed in the Core and Atomic variants indicates that removing the **demand** and **remaining_capacity** variables was counterproductive, as they appear necessary to properly account for load-based constraints.

The greatest improvement can be seen in the Time Window variants. The Core and Atomic variants exhibited very good performance in this case, indicating the ability to

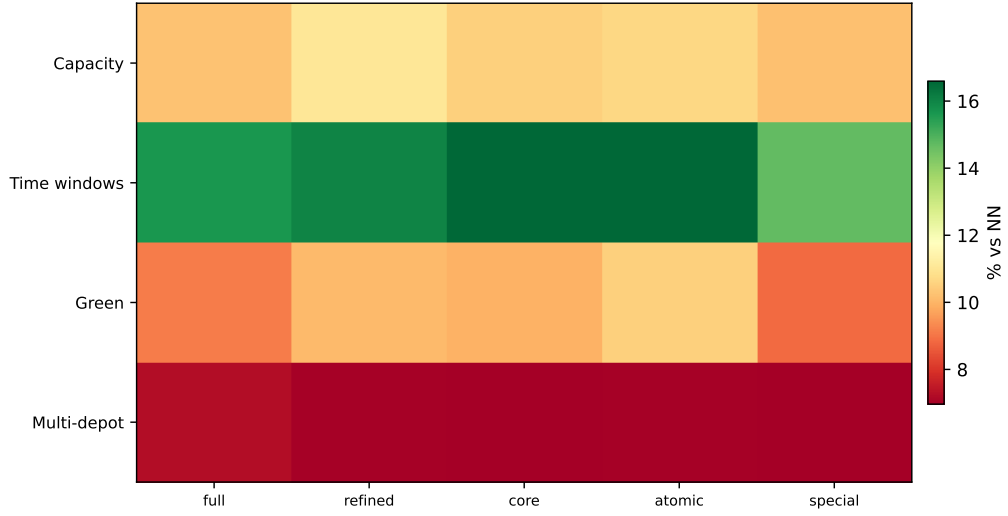


Figure 5.14: Mean percentage improvement over NN categorized by problem constraint.

develop high-quality timing logic with just three basic temporal terminals (`wait_time`, `arrival_time`, and `due_time`).

Green variants, on the other hand, exhibited their maximum performance in the Atomic variant. This indicates that `energy_to_customer` alone is sufficient to develop a robust energy management logic, while the other battery terminals provide little additional benefit.

Finally, performance in Multi-depot scenarios remained low. While the Full set demonstrated a slight advantage, the overall results indicate that the current terminal selection is inadequate for solving problems of this type. Future improvements in this category may require either implementing more specialized depot terminals beyond `depot_rank` and `depot_distance_advantage`, or a fundamental upgrade to the core route-construction logic.

Evolutionary Adaptation and Utilization

The reduction in search space led to a significant shift in terminal usage. Figure 5.15 illustrates its frequency across the different configurations.

As the set size is restricted, the utilization frequency of the remaining terminals increases. This was most evident in the atomic configuration, where some primitives, such as `dist_from_current` and `savings`, had their frequencies doubled compared to the full configuration.

5.4 GP Heuristic Performance

The following section compares the performance of the best GP heuristic rules against the NN baseline. The findings in Table 5.3 are aggregated from the results of all previously conducted experiments.

It can be concluded that evolved heuristic rules are superior to the baseline NN approach across all problem variations considered. The largest improvements are achieved for problems with temporal constraints. For instance, the VRPTW variant achieves the highest cost reduction of 33.7% in the unconstrained version and 29.7% with capacity limits. This

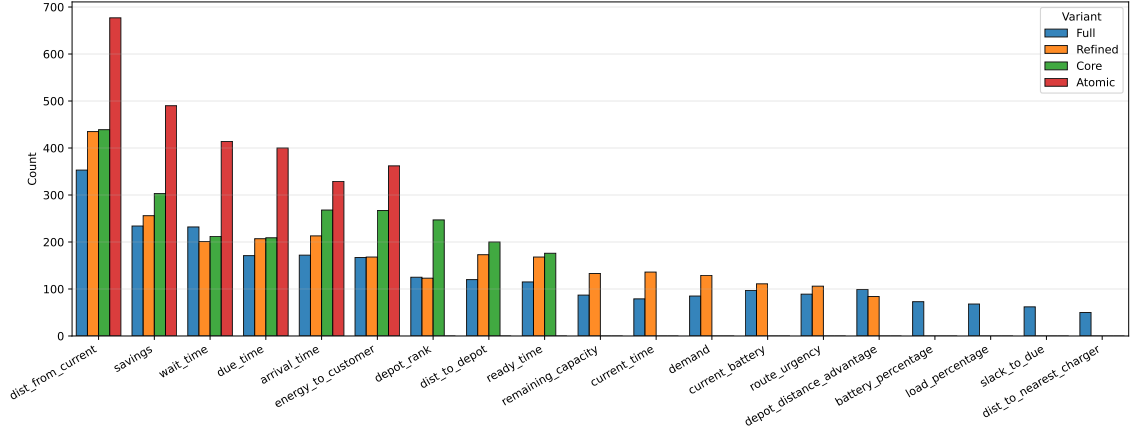


Figure 5.15: Terminal utilization counts across different set configurations.

proves that GP can evolve complex strategies to balance the total travel cost with severe arrival deadlines.

Table 5.3: Performance Comparison: Aggregated Best GP Results vs. NN

Idx	Type	Cap	GP Avg	NN Avg	% vs NN
1	VRP	N	617.85	687.59	+10.1%
2	MDVRP	N	2,603.86	2,919.82	+10.8%
3	GVRP	N	556.91	626.10	+11.1%
4	GVRP-MD	N	589.28	612.38	+3.8%
5	VRPTW	N	6,230.76	9,400.97	+33.7%
6	MDVRPTW	N	3,458.32	4,013.05	+13.8%
7	G-VRPTW	N	1,062.97	1,326.07	+19.8%
8	G-VRPTW-MD	N	919.94	1,102.25	+16.5%
9	VRP	Y	1,277.20	1,453.26	+12.1%
10	MDVRP	Y	3,456.32	3,852.94	+10.3%
11	GVRP	Y	703.39	780.87	+9.9%
12	GVRP-MD	Y	676.42	733.88	+7.8%
13	VRPTW	Y	6,684.00	9,509.66	+29.7%
14	MDVRPTW	Y	3,540.49	4,090.12	+13.4%
15	G-VRPTW	Y	1,071.53	1,326.07	+19.2%
16	G-VRPTW-MD	Y	925.19	1,102.25	+16.1%

Despite reduced performance gains due to capacity constraints, the GP still maintains a double-digit advantage in most cases. In the case of multi-depot problems, the performance gap to the baseline decreases significantly, especially for the MDVRPTW problem, where the gain is only 13.8% compared to 33.7% observed in the single-depot VRPTW.

The lowest improvement was observed in the unconstrained GVRP-MD variant (3.8%), suggesting that greedy heuristics are already relatively efficient for multi-depot Green routing problems without temporal or load pressures. This leaves less room for the GP to discover logic that significantly improves the routes.

Figure 5.16 illustrates the convergence behavior of the experiment that produced the best-performing heuristic for the uncapacitated VRPTW. The fitness drops sharply in the first 30 generations, then gradually plateaus.

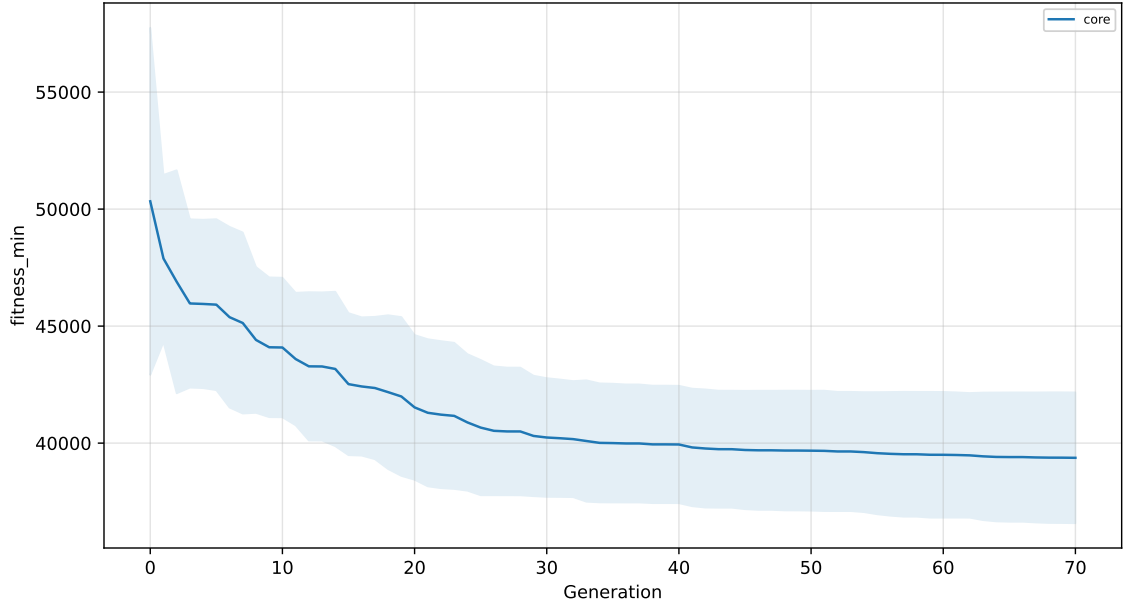


Figure 5.16: Mean best-per-generation fitness across five runs for the uncapacitated VRPTW (shaded: ± 1 std).

The most efficient heuristic discovered for the uncapacitated GVRP-MD consists of only 13 nodes, prioritizing the current distance while adjusting for battery levels and depot proximity. The tree structure is visualized in Figure 5.17. This is the smallest of the 16 best variants. For comparison, examples of other small best-performing trees are available in Appendix B.1.

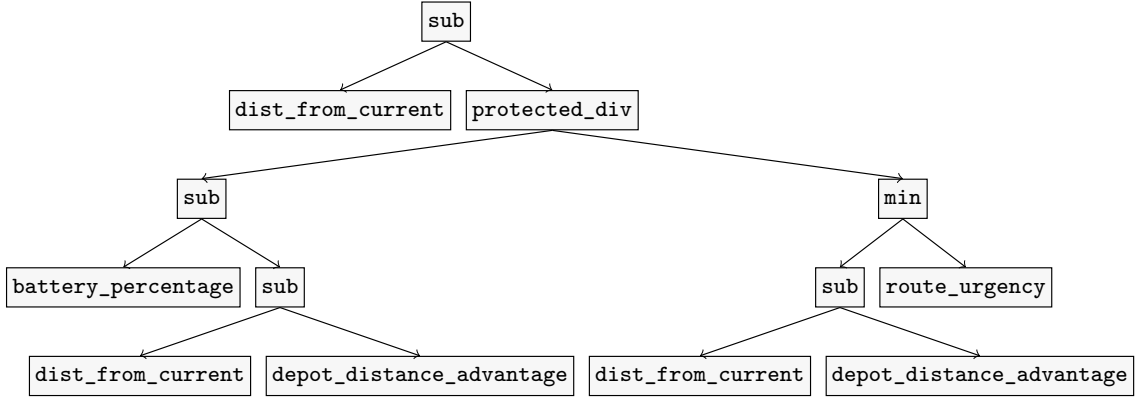


Figure 5.17: Tree structure of the best individual for the uncapacitated GVRP-MD.

The tree encodes the following priority function (function names are shortened):

$$\text{dist_from_c} - \frac{(\text{battery_perc} - (\text{dist_from_c} - \text{depot_distance_advantage}))}{\min(\text{dist_from_c} - \text{depot_distance_advantage}, \text{route_urgency})} \quad (5.2)$$

The root subtraction establishes distance from the current position as the base scoring criterion. This is adjusted by a normalized term that accounts for both remaining battery level and depot efficiency. The repeated appearance of `dist_from_current` –

`depot_distance_advantage` in both the numerator and denominator suggests the heuristic consistently weighs candidate distance against depot efficiency, which makes intuitive sense in a multi-depot setting. The role of `route_urgency` within the `min` operator is difficult to interpret and is likely to have emerged by chance during evolution rather than encoding any meaningful routing logic.

An investigation into the structural properties of the 16 winning heuristics reveals that they do not deviate significantly from the broader experimental population in terms of size or terminal distribution. The champion rules maintain an average size and depth consistent with the overall mean of the evolutionary runs.

Terminal usage across the 16 best-performing rules also largely mirrors the trends observed across all experimental runs. This uniformity across the „champion“ subset confirms that the evolutionary pressure consistently drives the population toward a specific type of routing logic, regardless of whether a particular individual eventually becomes the overall best performer.

5.4.1 Function and Constant Composition

Function usage follows a downward linear trend, with `add` being the most popular operator, followed by `max`, `sub`, and `min`. `protected_div` and `mul` were the least frequent, appearing at roughly one-third the rate of `add`. This suggests a preference for additive accumulation and thresholding over ratio-based metrics.

Regarding constants, 1.0 was utilized three times as often as -1.0 or 0.0, which shared nearly identical frequencies. This indicates that the evolved logic relies heavily on unit increments and identity elements rather than corrective parity or nullification.

5.5 Heuristic Generation using Large Language Models

In addition to using GP, this thesis introduces a comparative benchmark by utilizing LLMs to develop priority functions for the routing variants identified in Table 4.2.

Whereas GP uses evolutionary search to find heuristics, LLMs rely on high-dimensional semantic patterns and logical reasoning. This allows comparison between evolved logic, learned through numerous iterative evaluations, and reasoned logic, synthesized from linguistic and structural patterns in large-scale training data. By using the same primitive and terminal sets in both methods, we can evaluate the effectiveness of the search mechanism itself and compare the pressure of natural selection with the capabilities of a transformer-based agent.

5.5.1 Model Selection

The model selected for the heuristic generation task was **Trinity-Large-Preview**, developed by **Arcee AI** [2] and accessed using the OpenRouter Application Programming Interface (API) [21] in March 2026. At the time of the experiment, Trinity-Large was identified as the highest-performing free-tier model on the OpenRouter platform, ranking #20 in the programming category.

Trinity-Large is a large-scale, open-weight language model. The model’s architecture uses a sparse Mixture-of-Experts (MoE) design with 400B parameters. The Trinity-Large model also uses a 4-of-256 expert routing strategy to ensure high reasoning throughput, resulting in approximately 13B active parameters per token. [2].

The model also possesses an important property: unlike generic conversational models, it was trained to handle complex toolchains and long, constraint-filled prompts. This makes Trinity-Large particularly suited for generating valid G-VRPTW-MD heuristics, as the task requires maintaining strict structural integrity while utilizing only the available terminals and functions [2].

The architecture natively supports an extensive context window of up to 512k tokens, though the Preview API used in this study employed a 128k window with 8-bit quantization [21]. This combination of a MoE architecture, high programming rank, and a focus on constraint-heavy prompting makes Trinity-Large an ideal candidate for comparison with the evolutionary results.

5.5.2 Prompt Engineering

To ensure the LLM understood its goal and available resources, a highly structured system prompt was created. Due to its length, the full text is included in Appendix A. The system prompt was divided into several parts designed to assist the LLM in achieving a successful solution:

- **Instructional Goal:** The prompt defined the LLMs goal of creating a priority function for a route-construction algorithm where a lower score indicates a higher preference for the next customer node.
- **Variant Context:** A dynamic description of the specific routing constraints (e.g., Green VRP, Time Windows, or Multi-Depot) for the chosen variant.
- **Available Features:** A list of terminal variables specific to the selected variant, along with short descriptions. This information is identical to that in Section 4.4.1.
- **Constants and Operations:** A set of available constants (0.0, 1.0, -1.0) and operations, as discussed in Section 4.4.2, was introduced.
- **Structural Examples:** To familiarize the LLM with the required syntax, a reference function, such as `add(demand, protected_div(...))`, was included.
- **Output Requirements:** The system prompt included strict formatting guidelines, ensuring that the LLM generated a single-line function with a specific syntax to be parsed by the evaluation script.

5.5.3 Candidate Selection

To mitigate the inherent stochasticity of LLM outputs, an automated pipeline was developed to perform a “Best-of- N ” sampling strategy rather than relying on a single inference. Specifically, for each problem variant presented in this thesis (see Table 4.2), the pipeline executed 50 independent API requests. By using multiple samples, we accounted for variability in the model responses and increased the likelihood of finding a high-performing heuristic for each variant.

From the 800 total candidates (50 samples $\times$ 16 variants), a screening and selection process was used to identify a representative “champion” heuristic for each problem variant. Specifically, we employed a two-part selection process based on the following criteria:

1. **Syntactic Validity:** First, all of the invalid heuristics had to be discarded. This was done by running the route-construction process for each of them and checking for any errors.
2. **Empirical Performance:** From all the remaining valid candidates, the heuristic that achieved the highest performance, specifically had the lowest distance traveled across all of the instances, was selected as the final representative for that variant.

This selection ensures that the LLM baseline represents the model’s best attempt for each specific problem configuration, enabling us to compare it with the evolved GP heuristics.

5.5.4 Results

Validity of the Results

The results’ syntax was acceptable, as almost all API calls were answered correctly. There were a few examples of infinite loops, where the model produced a continuous chain of addition operations (e.g., `add(add(add(...)))`). This happened in 40 of the 800 cases. Sometimes the model started its answer with unnecessary text, as if it was creating a Python function utilizing `def` and `return`, this happened in 106 cases, but these ones weren’t discarded, just fixed.

The most significant deviation from the prompt involved the use of constants; the model often utilized ones it did not have available, according to the prompt. This mostly concerned the constant 0.1, but also others. As unavailable numbers appeared in approximately 180 of the provided responses, I decided to retain the original numerical outputs and use them within the simulation environment, rather than discard them. This decision was made to preserve the model’s intended logic.

Heuristic Size

Performing an analysis on the length of generated heuristics reveals a distinct preference for simplicity within the LLM baseline. Across the total pool of 756 parsed and valid candidate rules, the heuristic size (measured by the number of nodes/tokens in the expression) ranged from 5 to 55, with a mean of 13.18. However, when isolating the 16 “champion” heuristics, the complexity significantly decreases and stabilizes, featuring a mean size of 10.38 and a maximum of only 13 (see Table 5.4).

Table 5.4: Comparative Size Statistics for LLM-Generated Heuristics

Candidate Group	Min Size	Median	Mean	Max Size
Total Pool ($N = 756$)	5	13.0	13.23	55
Top 16 Champions	9	10.0	10.38	13

This data indicates a strong inverse correlation between complexity and performance in the LLM outputs. While the model occasionally generated larger, more intricate expressions (up to 55 tokens), these candidates consistently yielded poor results in the simulation environment. This implies that the LLM cannot sustain logical consistency in highly nested priority functions. On the other hand, the best rules were those that remained compact, typically involving only three or four primary features.

Compared with the LLM baseline, the heuristics evolved by GP display a significantly higher degree of structural complexity. While the LLM champions remained compact, with a mean size of 10.38 nodes, the best-performing GP individuals frequently used between 50 and 200 nodes, with some outliers exceeding 400. Unlike the LLM, which shows an inverse correlation between complexity and performance, the GP uses structural growth to discover more sophisticated logic. These findings suggest that the LLM is restricted by a complexity ceiling that favors simplicity, because of this, the LLM rules are smaller and easier to interpret.

Figure 5.18 illustrates the LLM rule size distribution.

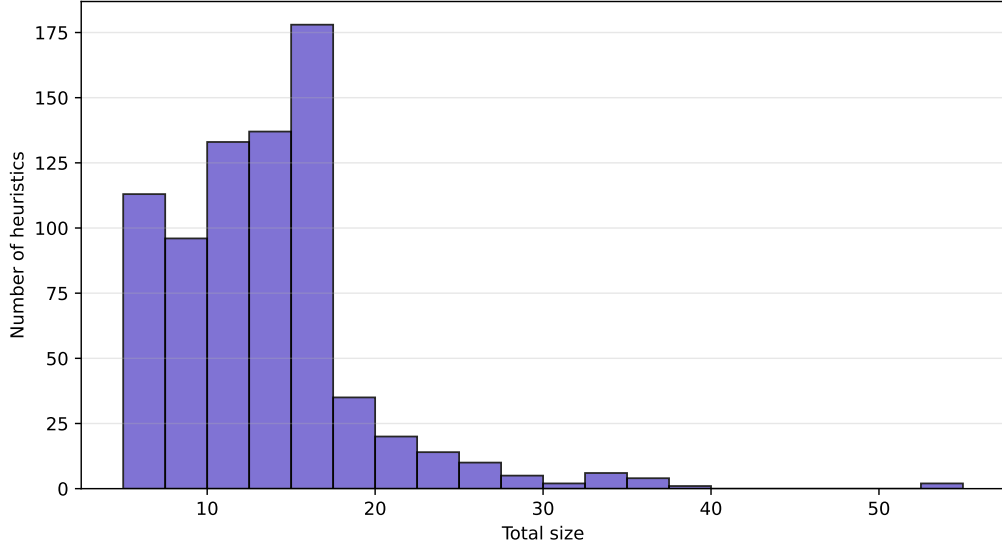


Figure 5.18: Distribution of heuristic sizes for all valid LLM candidates.

Utilization of Terminals

Figure 5.19 illustrates the total count of each terminal appearing in the generated heuristics across all problem variants. These frequencies are influenced by the availability of each feature: while `dist_from_current`, `savings`, and `dist_to_depot` are present in all 16 variants, the other terminals appear only in 50% of cases where their associated problem constraints are enabled. Despite this availability bias, the overwhelming frequency of `dist_from_current`, which appears in over 700 instances, indicates that the model consistently prioritizes basic Euclidean distance regardless of the specific problem constraints. Comparison between GP and LLM:

- **Spatial Metrics:** Both methods rely heavily on `dist_from_current` and `savings`, but the LLM utilizes `dist_from_current` at a rate approximately double that of the GP. The LLM also uses the `dist_to_depot` much more, ranking it as its third-most-used terminal.
- **Constraint Handling:** The GP evolution emphasizes temporal logic (`wait_time`, `arrival_time`), while the LLM prioritizes capacity (`remaining_capacity`, `demand`). This is a major departure from GP heuristics, where capacity terminals were candidates for pruning due to negligible use.

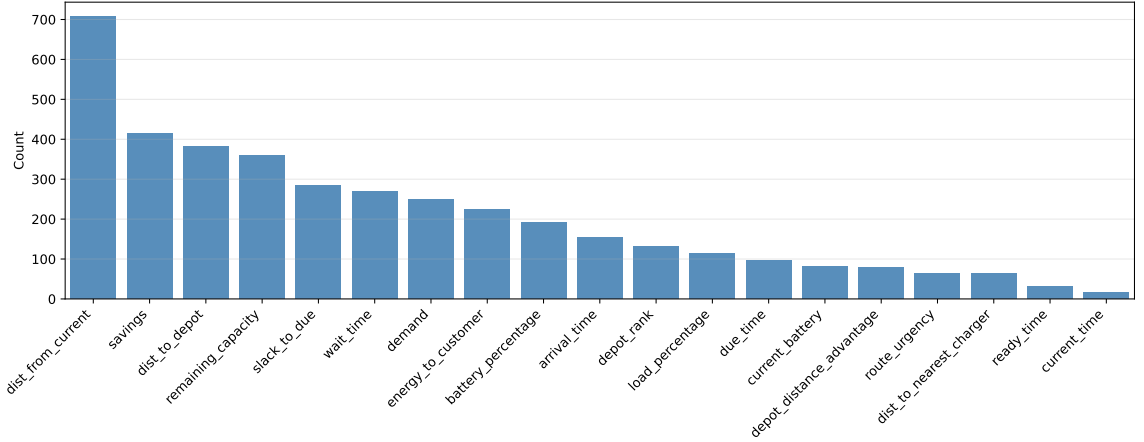


Figure 5.19: Frequency of Terminal Usage in LLM answers

- **Information Abstraction:** The LLM shows a preference for the `slack_to_due` terminal, which is largely ignored by the GP. This indicates that the LLM prioritizes “look-ahead” metrics to satisfy temporal constraints.

Function and Constant Usage

For the functions, the distribution of operations was skewed toward `protected_div`. This preference for division indicates that the model attempts to construct heuristics based on efficiency ratios (e.g., cost-per-unit-demand) rather than simple linear combinations.

Of the available constants, 1.0 was used most frequently, followed by 0.0. Notably, the permitted constant -1.0 appeared in only approximately 30 instances, whereas the unauthorized values 0.1 and 0.01 occurred far more frequently.

Performance Comparison

The comparison between the most successful heuristics from LLM-generated algorithms and the NN baseline is shown in Table 5.5. The results indicate that although an LLM can generate adequate logic for certain problem types, its efficiency depends significantly on the specific constraints of the routing variant.

A significant limitation revealed by the data is that the LLM failed to outperform the basic VRP baseline, trailing by -0.3% in unconstrained scenarios and -5.0% with capacity constraints. This result implies that, for strictly spatial problems, the algorithm fails to generate any new logic improvements beyond those of the greedy strategy. Ultimately, without the pressure of temporal or other constraints, the LLM mostly just recreates a suboptimal version of the standard nearest-neighbor logic.

The most prominent benefits have been observed in temporal cases with no capacity restrictions. The VRPTW and G-VRPTW models achieved improvements of 25.0% and 14.5%, respectively. Showcasing that the LLM successfully detects priorities among time-window terminals based on their spatial proximity. However, this advantage diminishes or even disappears in capacitated scenarios. The gain for VRPTW reduces to 11.6%, and the G-VRPTW performance falls below the baseline to -13.0%.

The LLM struggled most in multi-depot variants (MDVRP), showing a significant performance deficit of up to -29.2% against the NN algorithm. This result confirms the concept

Table 5.5: Performance Comparison: LLM vs. NN Baseline

Idx	Variant	Cap	LLM Avg	NN Avg	% vs NN
1	VRP	N	689.42	687.59	-0.3%
2	MDVRP	N	3,628.16	2,919.82	-24.3%
3	GVRP	N	603.48	626.10	+3.6%
4	GVRP-MD	N	629.40	612.38	-2.8%
5	VRPTW	N	7,052.07	9,400.97	+25.0%
6	MDVRPTW	N	3,957.63	4,013.05	+1.4%
7	G-VRPTW	N	1,133.75	1,326.07	+14.5%
8	G-VRPTW-MD	N	1,065.63	1,102.25	+3.3%
9	VRP	Y	1,525.25	1,453.26	-5.0%
10	MDVRP	Y	4,978.21	3,852.94	-29.2%
11	GVRP	Y	753.59	780.87	+3.5%
12	GVRP-MD	Y	719.94	733.88	+1.9%
13	VRPTW	Y	8,401.84	9,509.66	+11.6%
14	MDVRPTW	Y	4,515.55	4,090.12	-10.4%
15	G-VRPTW	Y	1,498.38	1,326.07	-13.0%
16	G-VRPTW-MD	Y	1,089.05	1,102.25	+1.2%

of a „complexity ceiling“ observed in the structural analysis: because the LLM favors compact rules, it fails to account for the additional logical layers required to manage multiple depots and other complex constraints. Although this approach remains applicable to temporal routing, the results suggest a limitation in the model’s ability to work with highly constrained search spaces.

5.6 Final Evaluation

The final results for each implemented route generation scheme are summarized in Table 5.6 and visualized in a radar chart 5.20.

Comparing the performance across all sixteen tested VRP variants clearly shows differences in the effectiveness of each approach and illustrates how each heuristic scales with constraints such as capacity, time windows, and multiple depots. The evaluated heuristics are divided into two construction scheme types: GP, LLM, and NN operate as Generation Constructive approaches, while CW Savings represents the Selection Constructive type [20].

Among the three Generation Constructive heuristics, GP achieved the best results overall. Its advantage was most evident in time-windowed variants, where the gap over the remaining approaches widened considerably. For example, in VRPTW-N, GP scored 6,230.76, while NN reached 9,400.97, resulting in a nearly 34% difference in total cost. This showcases GP’s capability to learn routing policies that incorporate temporal feasibility, a property that greedy constructive heuristics cannot capture by design. The LLM-generated heuristic fell short of GP in every tested variant, with the gap widening as structural complexity increased. The three variants where GP outperformed all other heuristics, including the CW Savings, were GVRP-N (556.91 vs. 606.45), G-VRPTW-N (1,062.97 vs. 1,106.76), and G-VRPTW-Y (1,071.53 vs. 1,107.13), all of which belong to the range-constrained green variants where vehicles must visit charging stations before their battery is depleted. This suggests that GP was able to evolve a scoring logic that implicitly accounts for the remain-

Table 5.6: Performance Comparison: GP, LLM, NN, and CW Savings

Idx	Variant	Cap	GP Avg	LLM Avg	NN Avg	Savings Avg
1	VRP	N	617.85	689.42	687.59	603.00
2	MDVRP	N	2,603.86	3,628.16	2,919.82	2,318.15
3	GVRP	N	556.91	603.48	626.10	606.45
4	GVRP-MD	N	589.28	629.40	612.38	572.09
5	VRPTW	N	6,230.76	7,052.07	9,400.97	5,986.77
6	MDVRPTW	N	3,458.32	3,957.63	4,013.05	3,299.57
7	G-VRPTW	N	1,062.97	1,133.75	1,326.07	1,106.76
8	G-VRPTW-MD	N	919.94	1,065.63	1,102.25	757.92
9	VRP	Y	1,277.20	1,525.25	1,453.26	1,094.05
10	MDVRP	Y	3,456.32	4,978.21	3,852.94	2,597.10
11	GVRP	Y	703.39	753.59	780.87	657.35
12	GVRP-MD	Y	676.42	719.94	733.88	588.77
13	VRPTW	Y	6,684.00	8,401.84	9,509.66	6,095.62
14	MDVRPTW	Y	3,540.49	4,515.55	4,090.12	3,302.71
15	G-VRPTW	Y	1,071.53	1,498.38	1,326.07	1,107.13
16	G-VRPTW-MD	Y	925.19	1,089.05	1,102.25	758.02

ing range when selecting the next node, something that neither greedy insertion nor the fixed savings formula can do inherently.

The LLM-generated heuristic produced mixed results, matching the NN overall but exhibiting a different performance curve. It outperformed NN in 9 of the 16 tested variants, with its advantage concentrated in time-windowed and green variants, in VRPTW-N it achieved 7,052.07 against NN’s 9,400.97, and similar improvements appeared in MDVRPTW-N, G-VRPTW-N, and G-VRPTW-MD-N. This behavior follows the same pattern observed for GP, suggesting that the LLM-generated function captures some awareness of temporal and range feasibility, although not nearly as successfully as the evolved GP heuristics. On the other hand, NN outperformed the LLM in 7 variants, mostly in variants with multi-depot structure or without time windows, such as MDVRP-N, MDVRP-Y, and MDVRPTW-Y, where spatial proximity leveraged by NN proved more effective than the LLM’s general routing logic. As mentioned before, against the GP the LLM trailed in every variant, and in complex cases such as G-VRPTW-Y the gap reached 40% (1,498.38 vs. 1,071.53). As the LLM heuristic was generated from a single prompt with no iterative refinement or fitness feedback, it relies solely on routing intuition and lacks the targeted optimization achieved through evolutionary search.

The NN heuristic performed well in some variants but turned out to be extremely sensitive to the nature of the applied constraints. In simpler spatial problems like VRP and GVRP-MD, it showed satisfactory performance, while in multi-depot situations it clearly outperformed the LLM approach. With time windows introduced, the key weakness of the nearest-insertion strategy became obvious, as no assessment of temporal feasibility was possible during solution construction. As a result, its efficiency dropped dramatically compared to both other algorithms. In VRPTW-N NN reached 9,400.97 against GP’s 6,230.76 and CW’s 5,986.77, and the pattern held across MDVRPTW, G-VRPTW, and G-VRPTW-MD. Here, it showed lower efficiency not only than GP and CW but also than LLM. For lightly constrained variants, NN serves as a decent trivial benchmark, but it is clearly limited by the nature of the problem.

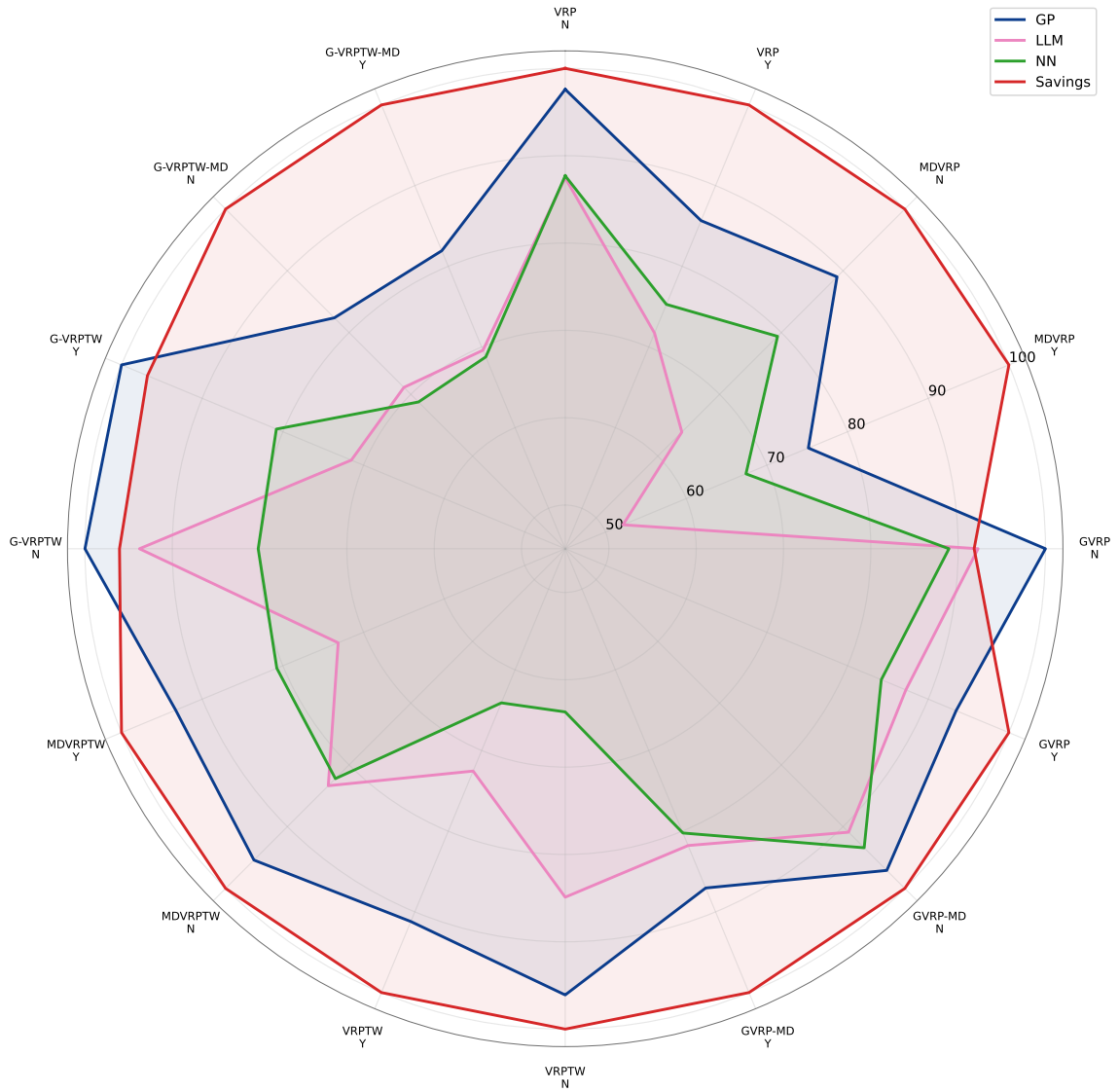


Figure 5.20: Normalized performance comparison of all heuristics

The CW Savings algorithm differs from the other three approaches fundamentally in its construction scheme, which is why it is treated here as a reference benchmark rather than a direct competitor. While GP, LLM, and NN all operate as Generation Constructive schemes, ranking and adding candidate nodes one at a time through a scoring function, Savings utilizes a Selection Constructive method by merging pairs of routes based on a fixed savings formula. This distinction has practical implications beyond the difference in solution quality: because Generation Constructive schemes evaluate candidates individually at each step, they can respond to real-time changes such as updated constraints or new customer requests without having to remake the entire solution, whereas the merging logic of Savings assumes the problem is fully known upfront [20]. Despite this difference in construction logic, the numerical results are still worth examining in detail.

CW Savings achieved the lowest costs in 13 of the 16 tested variants, with its advantage over the primary heuristics generally growing in the capacitated settings. In MDVRP-Y,

for instance, the gap over GP reached 33% (2,597.10 vs. 3,456.32), compared to 12% in the uncapacitated variant, suggesting that capacity constraints expose limitations in the scoring-based approaches more than in the merging-based one. The cost advantage of CW Savings over GP varies from minimal in simple cases like VRP-N (603.00 vs. 617.85) to significant in multi-depot scenarios such as MDVRP-Y (2,597.10 vs. 3,456.32), with the merging logic being especially effective in handling distributed depot structures. As mentioned in the GP section, Savings was outperformed in GVRP-N, G-VRPTW-N, and G-VRPTW-Y, where a flexible scoring function performed better than a fixed merging scheme. Overall, the performance of CW Savings deteriorates more slowly than that of the other heuristics as problem complexity increases, reinforcing its value as a strong benchmark, whereas the Generation Constructive approaches, despite lacking in performance, offer flexibility that other approaches cannot match.

5.6.1 Optimal solutions

To evaluate the absolute performance of the evolved heuristics, GP results were compared against known optimal solutions. Benchmarks were available for only four of the five original, unaugmented datasets. Specifically, the G-VRPTW lacks precomputed optimal routes. As a result, this comparison is restricted to the capacitated VRP, MDVRP, VRPTW, and MDVRPTW variants.

Table 5.7: GP Performance Against Optimal Solutions

Variant	Cap	GP Avg	Optimal Avg	% vs Optimal
VRP	Y	1,277.20	1,041.93	-22.58%
MDVRP	Y	3,456.32	2,436.18	-41.87%
VRPTW	Y	6,684.00	4,474.97	-49.36%
MDVRPTW	Y	3,540.49	2,306.63	-53.49%

The results in Table 5.7 reveal a significant room for improvement across these variants. The VRP shows the smallest deficit of -22.58%, likely due to smaller average instance sizes in its original data. However, for all other variants the performance gap widens significantly, reaching -53.49% for the MDVRPTW. It is therefore evident that, despite the GP being able to identify generalized rules across the overall 16-variant search space, there remains considerable room to refine the heuristics which lack the precision required to compete with the efficiency of exact solvers.

Chapter 6

Conclusion

This thesis investigated the potential of HHs in automating the design of constructive routing heuristics for a diverse set of VRP variants. To accomplish this, a GPHH system was designed and implemented, capable of automatically evolving routing-priority functions across sixteen problem variants that combine capacity constraints, time windows, multiple depots, and electric vehicle range constraints. The evolved heuristics were evaluated against two baselines, the NN and CW Savings, as well as against heuristics generated using an LLM, where the model was prompted to produce a routing priority function similar to those evolved by GP, without any fitness feedback.

The results confirm that GPHH is a viable approach for automated heuristic design in vehicle routing. The evolved heuristics produced better results than both the NN baseline and the LLM-generated approach across all tested variants. The largest performance difference occurred in variants with time windows and range constraints, where greedy construction methods lack the foresight necessary for competitive results. In three of the sixteen variants, the GP-based heuristics surpassed even the CW Savings algorithm, which demonstrated itself as a highly effective and reliable benchmark. It should, however, be noted that, due to its Selection Constructive nature, the CW Savings algorithm is unsuitable for dynamic scenarios, where changes happen during route generation, giving the Generation Constructive approaches a natural advantage. Comparison against known optimal solutions revealed deficits ranging from 23% to 53%, suggesting that although the developed heuristics generalized effectively across various problem domains, considerable room for improvement remains.

The LLM experiment produced mixed results. While the LLM-generated heuristic outperformed NN in nine out of the sixteen variants, it fell short of GP in every single case, and in complex variants, the gap was substantial. This suggests that without iterative refinement or fitness feedback, LLM-generated heuristics capture only a surface-level understanding of routing constraints. Whether this approach could be developed into a more competitive alternative through iterative prompting or evolutionary post-processing is an interesting open question.

Given the strong performance of CW Savings, applying GP within a Selection Constructive framework to evolve its merging logic is a compelling research direction, potentially combining the solution quality of savings-based construction with the adaptability of evolutionary search.

Overall, this thesis demonstrates that GPHH is a practical and flexible alternative to manual heuristic engineering and becomes increasingly advantageous as problem complexity increases.

Bibliography

- [1] APPLEGATE, D. L.; BIXBY, R. E.; CHVÁTAL, V. and COOK, W. J. *The Traveling Salesman Problem: A Computational Study*. Princeton, NJ: Princeton University Press, 2006. ISBN 9780691129938.
- [2] ARCEE AI. *Trinity-Large-Preview: Efficiency-First Frontier Language Modeling*. 2024. Available at: <https://docs.arcee.ai/language-models/trinity-large-preview>. Official Technical Announcement.
- [3] AUGERAT, P.; BELENGUER, J. M.; BENAVENT, E.; CORBERÁN, A.; NADDEF, D. et al. *Computational Results with a Branch and Cut Code for the Capacitated Vehicle Routing Problem*. Research Report 949-M. Université Joseph Fourier, Grenoble, 1995.
- [4] BALDACCI, R.; MINGOZZI, A. and ROBERTI, R. Recent exact algorithms for solving the vehicle routing problem under capacity and time window constraints. *European Journal of Operational Research*. Elsevier, 2012, vol. 218, no. 1, p. 1–6.
- [5] BRAEKERS, K.; RAMAEKERS, K. and VAN NIEUWENHUYSE, I. The vehicle routing problem: State of the art classification and review. *Computers & Industrial Engineering*, 2016, vol. 99, p. 300–313. ISSN 0360-8352. Available at: <https://www.sciencedirect.com/science/article/pii/S0360835215004775>.
- [6] BURLACU, B.; YANG, K. and AFFENZELLER, M. Population diversity and inheritance in genetic programming for symbolic regression. *Natural Computing*, january 2023, vol. 23.
- [7] CHETAN CHUDASAMA, M. P. Comparison of Parents Selection Methods of Genetic Algorithm for TSP. *International Conference on Computer Communication and Networks CSI-COMNET-2011*. New York, USA: Foundation of Computer Science (FCS), NY, USA, Decmber 2011, COMNET, no. 1, p. 102–105. ISSN 0975-8887. Available at: </proceedings/comnet/number1/5431-1019/>.
- [8] CLARKE, G. and WRIGHT, J. W. Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*. INFORMS, 1964, vol. 12, no. 4, p. 568–581.
- [9] CORDEAU, J.-F.; GENDREAU, M. and LAPORTE, G. A tabu search heuristic for periodic and multi-depot vehicle routing problems. *Networks*, 1997, vol. 30, no. 2, p. 105–119. Available at: <https://onlinelibrary.wiley.com/doi/abs/10.1002/%28SICI%291097-0037%28199709%2930%3A2%3C105%3A%3AAID-NET5%3E3.0.CO%3B2-G>.
- [10] DANTZIG, G. B. and RAMSER, J. H. The truck dispatching problem. *Management Science*. INFORMS, 1959, vol. 6, no. 1, p. 80–91.

- [11] GEHRING, H. A Parallel Hybrid Evolutionary Metaheuristic for the Vehicle Routing Problem with Time Windows. In: 1999. Available at: <https://api.semanticscholar.org/CorpusID:15357452>.
- [12] GIL GALA, F. J.; DURASEVIC, M. and JAKOBOVIC, D. Evolving routing policies for electric vehicles by means of genetic programming. *Applied Intelligence*, December 2024, vol. 54, p. 12391–12419. Available at: <https://rdcu.be/dZ4aj>.
- [13] KHAYYA, E.; MEDARHRI, I. and ZINE, R. A survey of the vehicle routing problem and its variants: formulations and solutions. *Mathematical Modeling and Computing*. Lviv Polytechnic National University, 2024, vol. 11, no. 1, p. 333–343.
- [14] KONSTANTAKOPOULOS, G. D.; GAYIALIS, S. P. and KECHAGIAS, E. P. Vehicle routing problem and related algorithms for logistics distribution: a literature review and classification. *Operational Research*, Jul 2022, vol. 22, no. 3, p. 2033–2062. ISSN 1866-1505. Available at: <https://doi.org/10.1007/s12351-020-00600-7>.
- [15] KONSTANTAKOPOULOS, G. D.; GAYIALIS, S. P. and KECHAGIAS, E. P. Vehicle routing problem and related algorithms for logistics distribution: a literature review and classification. *Operational Research*. Springer, 2022, vol. 22, no. 3, p. 2033–2062.
- [16] LABADIE, N.; PRINS, C. and PRODHON, C. *Metaheuristics for Vehicle Routing Problems*. 1st ed. John Wiley & Sons, 2016. ISBN 978-1-84821-811-6.
- [17] LENSTRA, J. K. and RINNOOY KAN, A. H. G. *Complexity of Vehicle Routing and Scheduling Problems*. Econometric Institute Archives 272191. Erasmus University Rotterdam, Econometric Institute, 1979. Available at: <https://ageconsearch.umn.edu/record/272191>.
- [18] LIN, C.; CHOY, K.; HO, G.; CHUNG, S. and LAM, H. Survey of Green Vehicle Routing Problem: Past and future trends. *Expert Systems with Applications*, 2014, vol. 41, 4, Part 1, p. 1118–1138. ISSN 0957-4174. Available at: <https://www.sciencedirect.com/science/article/pii/S095741741300609X>.
- [19] MENDOZA, J. E.; GUÉRET, C.; HOSKINS, M.; LOBIT, H.; PILLAC, V. et al. (VRP-REP): The vehicle routing community repository. In: *Third Meeting of the EURO Working Group on Vehicle Routing and Logistics Optimization (VeRoLog)*. Oslo, Norway: [b.n.], 2014. Available at: <http://www.vrp-rep.org>.
- [20] NELISHIA PILLAY, R. Q. *Hyper-Heuristics: Theory and Applications*. Springer, 2018. ISBN 9783319965147.
- [21] OPENROUTER TEAM. *Arcee AI: Trinity Large Preview (Free) Model Card*. 2024. Available at: <https://openrouter.ai/arcee-ai/trinity-large-preview:free>. Accessed via OpenRouter API.
- [22] PESSOA, A.; SADYKOV, R.; UCHOA, E. and VANDERBECK, F. A generic exact solver for vehicle routing and related problems. *Mathematical programming*. Berlin/Heidelberg: Springer Berlin Heidelberg, 2020, vol. 183, 1-2, p. 483–523. ISSN 0025-5610.

- [23] PILLAC, V.; GENDREAU, M.; GUÉRET, C. and MEDAGLIA, A. L. A review of dynamic vehicle routing problems. *European Journal of Operational Research*, 2013, vol. 225, no. 1, p. 1–11. ISSN 0377-2217. Available at: <https://www.sciencedirect.com/science/article/pii/S0377221712006388>.
- [24] POLI, R.; LANGDON, W. and MCPHEE, N. *A Field Guide to Genetic Programming*. Lulu Enterprises, UK Ltd, january 2008. ISBN 978-1-4092-0073-4.
- [25] PSARAFTIS, H. N.; WEN, M. and KONTOVAS, C. A. Dynamic vehicle routing problems: Three decades and counting. *Networks*, 2016, vol. 67, no. 1, p. 3–31. Available at: <https://onlinelibrary.wiley.com/doi/abs/10.1002/net.21628>.
- [26] ROCHA LEITE, C. F. M. da. *Genetic Programming Approaches for Solving Transportation Problems*. Porto, Portugal, 2022. Master’s thesis. Faculdade de Engenharia da Universidade do Porto (FEUP). Available at: <https://hdl.handle.net/10216/144322>.
- [27] SCHNEIDER, M.; STENGER, A. and GOEKE, D. The Electric Vehicle-Routing Problem with Time Windows and Recharging Stations. *Transportation Science*. INFORMS, 2014, vol. 48, no. 4, p. 500–520. ISSN 00411655, 15265447. Available at: <http://www.jstor.org/stable/43666939>.
- [28] SOLOMON, M. M. Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints. *Operations Research*. INFORMS, 1987, vol. 35, no. 2, p. 254–265.
- [29] TOTH, P. and VIGO, D. *Vehicle Routing: Problems, Methods, and Applications*. Society for Industrial and Applied Mathematics, 2015. ISBN 16-119-7358-9.
- [30] WEN, M. *Rich Vehicle Routing Problems and Applications*. Dissertation. ISBN 978-87-90855-89-5.

Appendix A

LLM Generation Prompt

LLM Prompt Template

The following box contains the static template used for all 16 problem variants. The placeholders in curly braces were replaced dynamically by the generation script.

Generic Heuristic Generation Prompt

You are an expert in vehicle routing problems (VRP).

Create a scoring function for selecting the next customer in a VRP route construction algorithm.

Variant {variant_description}

Context:

- Lower score = better customer to visit next (minimization)
- Function will be called repeatedly to build routes sequentially

Available features (as function parameters): {features_list}

Constants available: 0.0, 1.0, -1.0

Operations available: add, sub, mul, protected_div, max, min

Requirements:

1. Function example: `add(demand, protected_div(sub(dist_from_current, travel_time), protected_div(dist_to_depot, dist_from_current)))`
2. Return a single float (the score)
3. Only use feature names that appear in the feature list above (do not invent new parameters).
4. Write the function on one line

Output ONLY the function, no explanation before or after.

Prompt Variables for Capacitated G-VRPTW-MD

For the most complex variant investigated in this study, the following values were passed into the template variables above.

A.0.1 Variant Description

G-VRPTW-MD: Multi Depot, Green Vehicle Routing Problem with capacity constraints and Time Windows

A.0.2 Available Features List

dist_to_depot: distance from (route) depot to candidate customer,
dist_from_current: distance from current node to candidate customer,
savings: Clarke-Wright style savings for inserting candidate,
demand: demand of candidate customer,
remaining_capacity: remaining vehicle capacity before adding candidate,
load_percentage: current load divided by vehicle capacity,
current_time: current time in route construction,
arrival_time: arrival time at candidate if selected next,
ready_time: earliest allowed service start time at candidate,
due_time: latest allowed service start time at candidate,
wait_time: waiting time before service can start at candidate,
slack_to_due: time left until due time at arrival,
route_urgency: slack_to_due divided by travel distance (higher = less urgent),
current_battery: current battery level of vehicle,
battery_percentage: current_battery divided by battery capacity,
energy_to_customer: energy required to reach candidate from current node,
dist_to_nearest_charger: distance from candidate to nearest charging station,
depot_rank: rank of this depot by distance to candidate (1 = closest depot),
depot_distance_advantage: how much closer this depot is vs second-closest depot to candidate

Appendix B

Examples of evolved trees

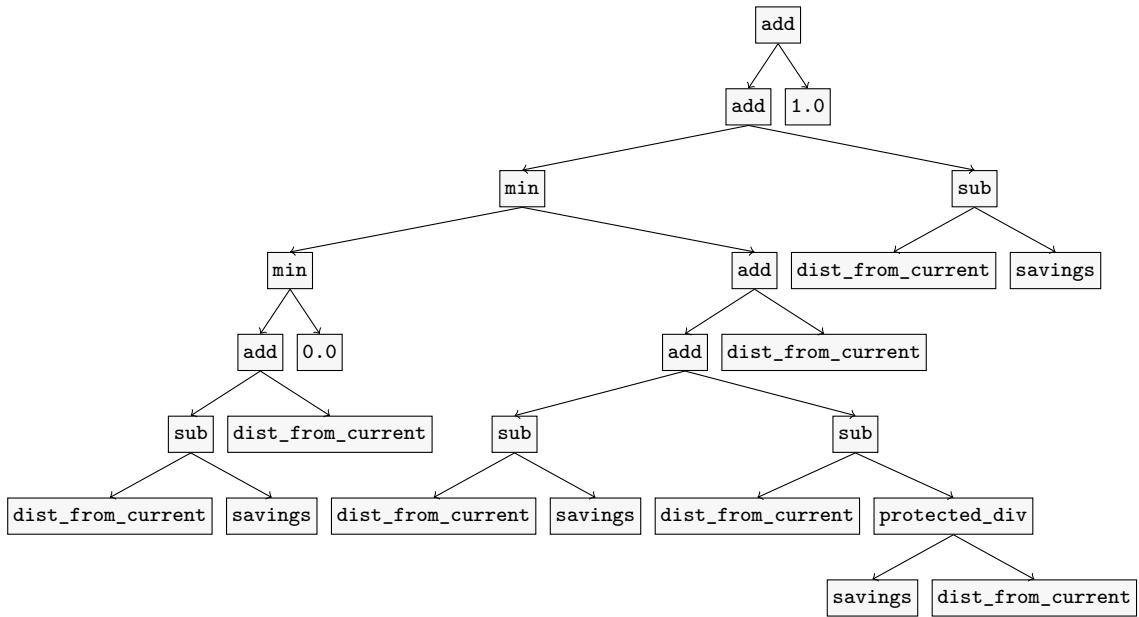


Figure B.1: Tree structure of the best-performing individual for uncapacitated VRP.

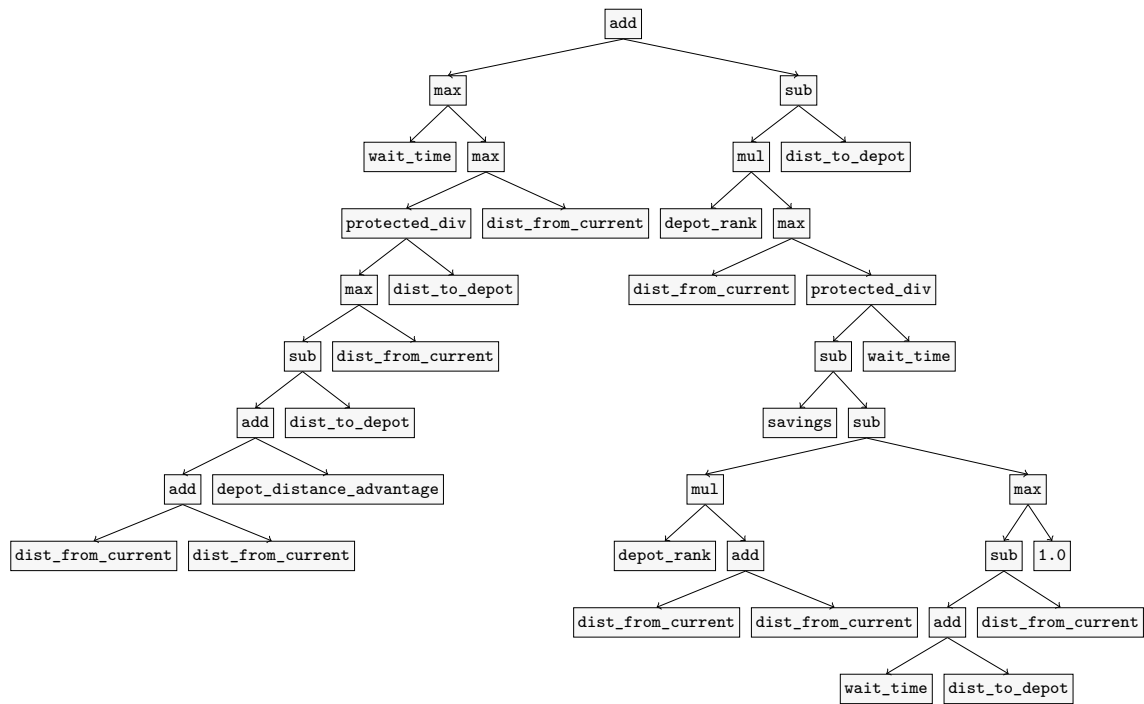


Figure B.2: Tree structure of the best-performing individual for uncapacitated MDVRPTW.

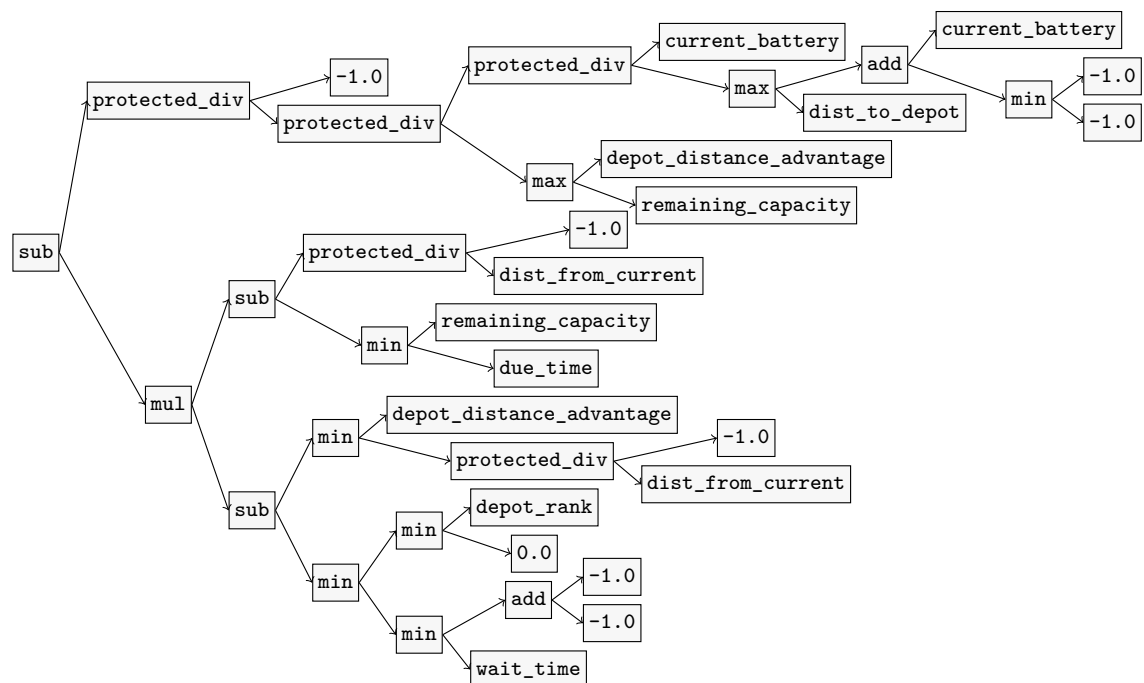


Figure B.3: Tree structure of the best-performing individual for capacitated MDGVRP.